



University of
Sheffield



A World
TOP 100
University



**UNIVERSITY
OF THE YEAR**

Search-Based Software Testing

Software Testing (COM3529)

Checkin code:

José Rojas & Neil Walkinshaw

Week 9

Roadmap

Beizer's Maturity Model

Test Automation

Unit Testing

Control / Data-Flow Analysis

Code Coverage

Regression Testing

Model-Based Testing

Mutation Testing

Search-Based Test Generation

Fuzzing

Roadmap

Beizer's Maturity Model

Test Automation

Unit Testing

Control / Data-Flow Analysis

Code Coverage

Regression Testing

Model-Based Testing

Mutation Testing

Search-Based Test Generation

Fuzzing

A hand holding a pen is positioned over a document, with a blue overlay covering the entire image. The text "What are we searching for?" is written in white, bold, sans-serif font across the center of the image.

**What are we searching
for?**

The Test Generation Challenge

Goal: Achieve branch coverage.

The Test Generation Challenge

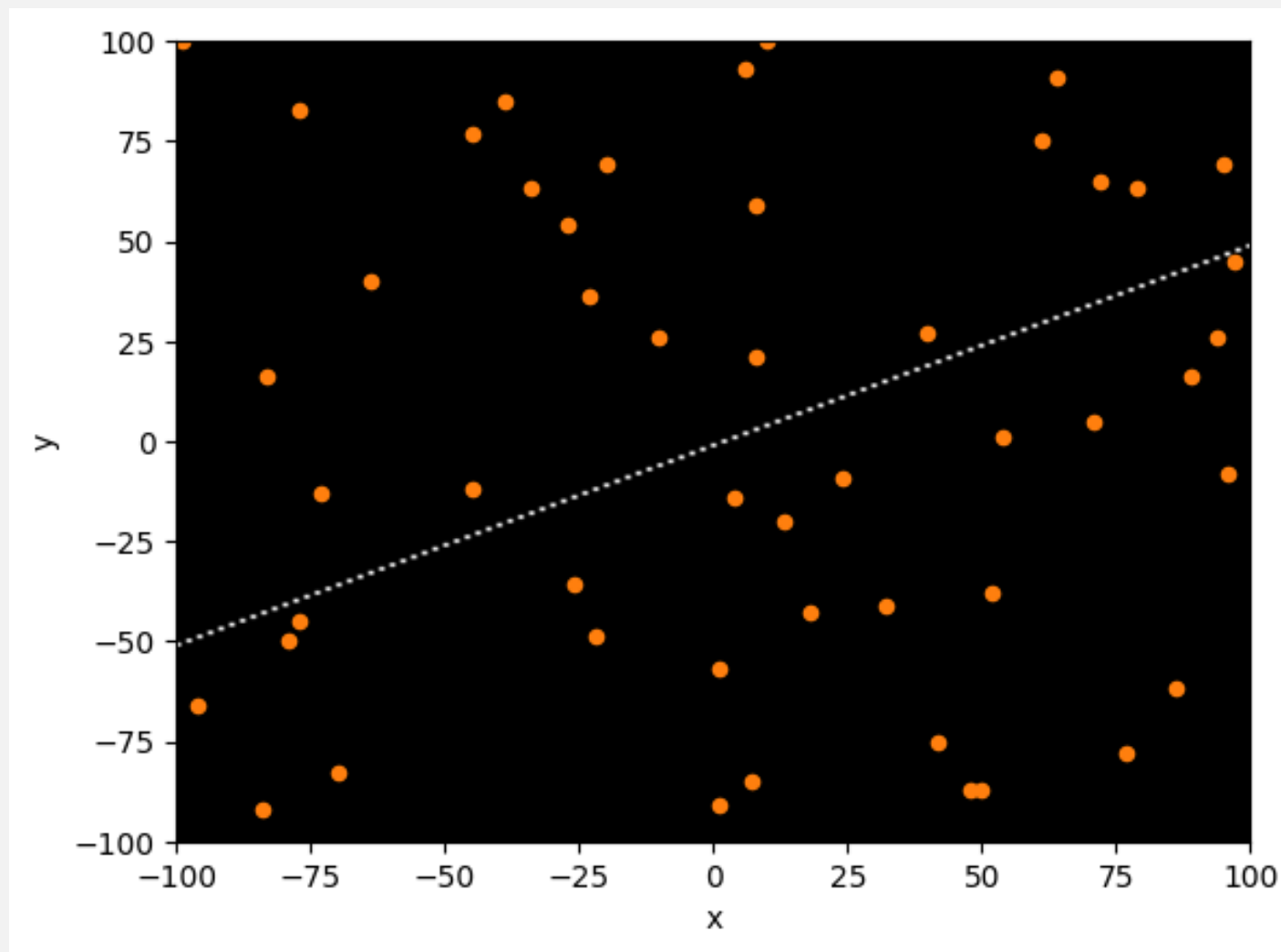
Goal: Achieve branch coverage.

```
def testMe(x, y):  
    if x == 2 * (y + 1):  
        return True  
    else:  
        return False
```

The Test Generation Challenge

Goal: Achieve branch coverage.

```
def testMe(x, y):  
    if x == 2 * (y + 1):  
        return True  
    else:  
        return False
```

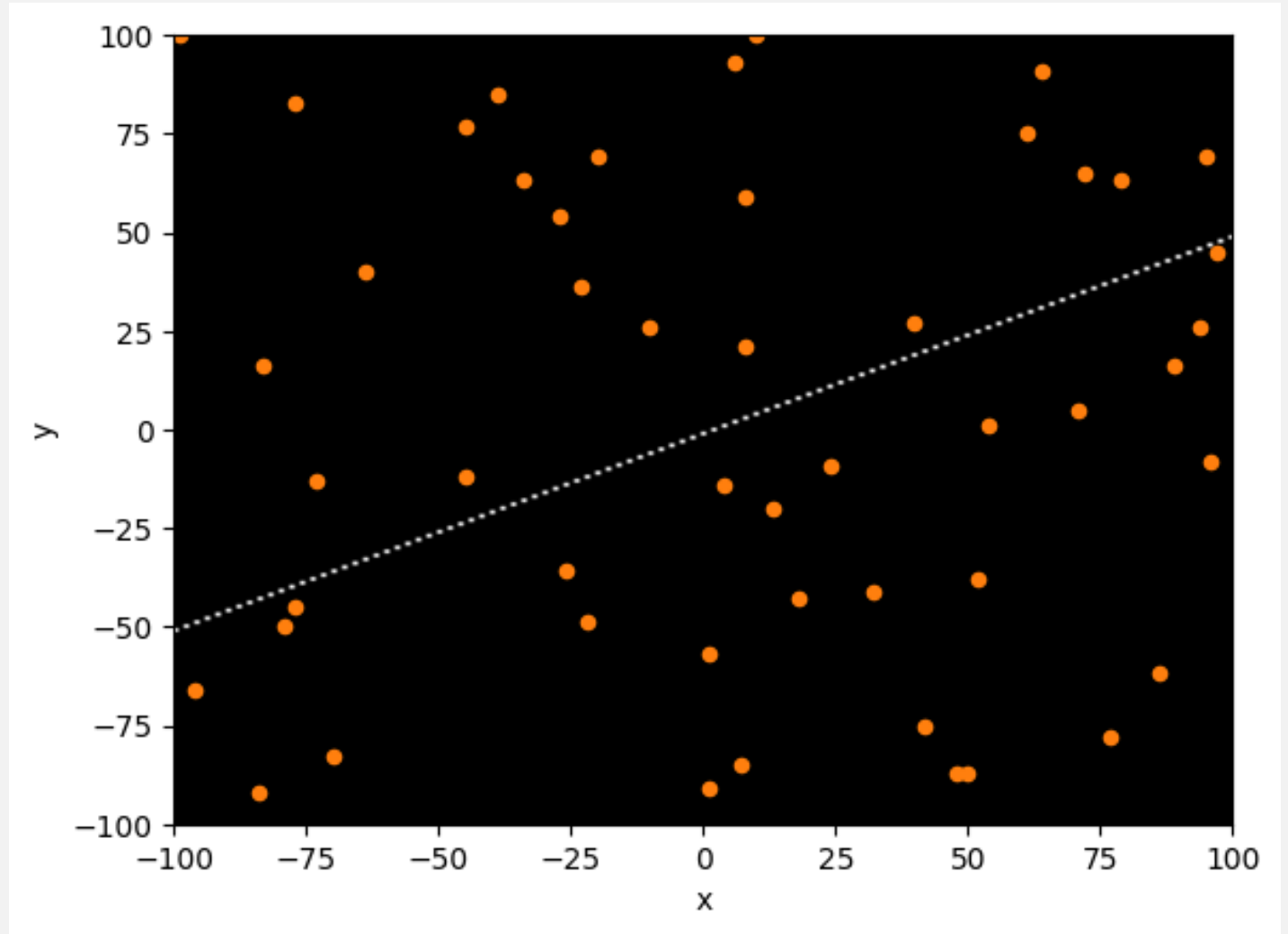


The Test Generation Challenge

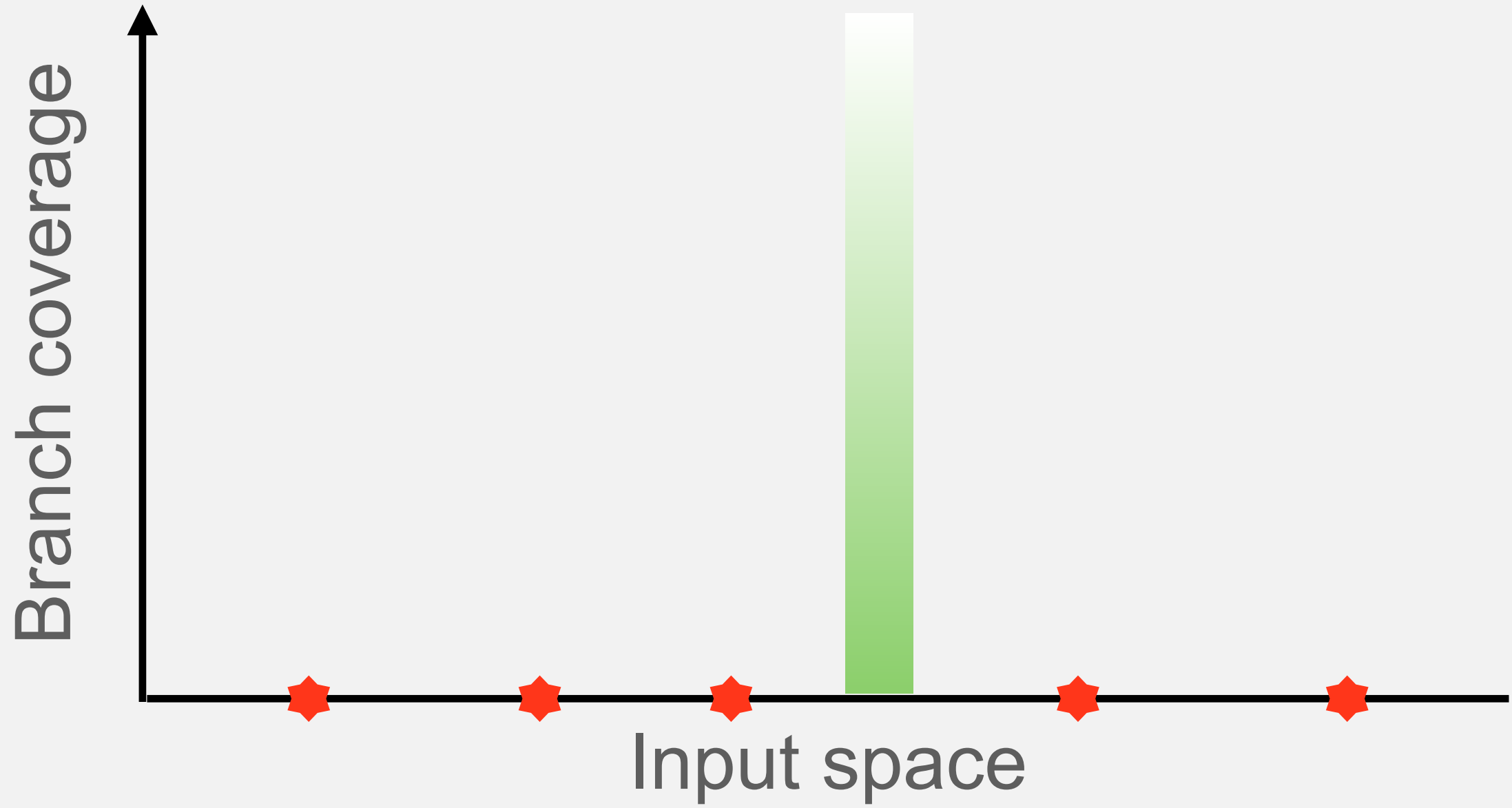
Goal: Achieve branch coverage.

```
def testMe(x, y):  
    if x == 2 * (y + 1):  
        return True  
    else:  
        return False
```

Random testing is very efficient.
Tend to trivially fail to hit the condition.



The essential challenge



Generating Tests can be Time Consuming

Certain levels of code coverage are often **mandated**.

- Industry standards for safety-critical systems.

- Policies within development organisations.

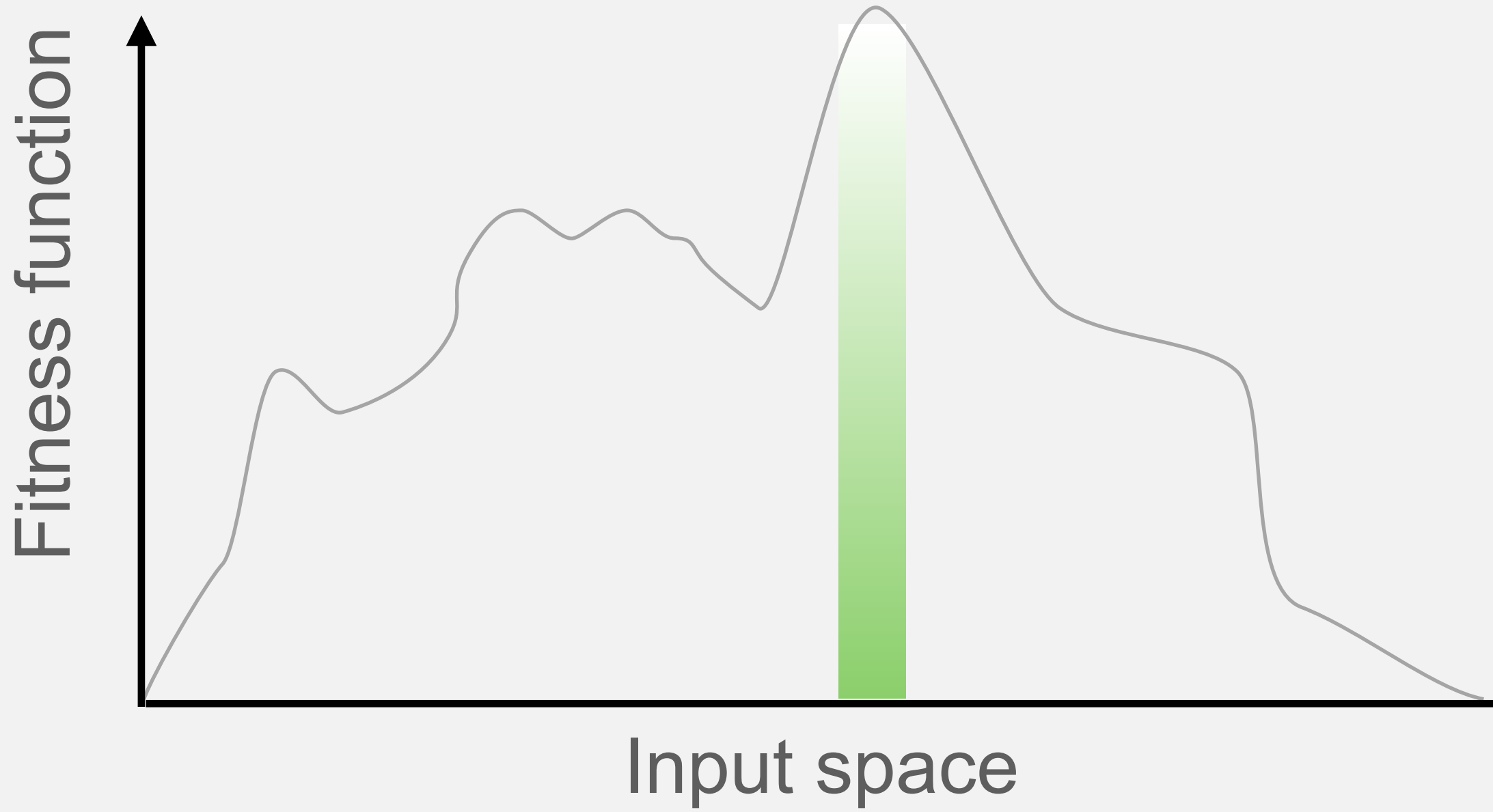
- ...

Finding the test sets to cover the necessary code can be challenging.

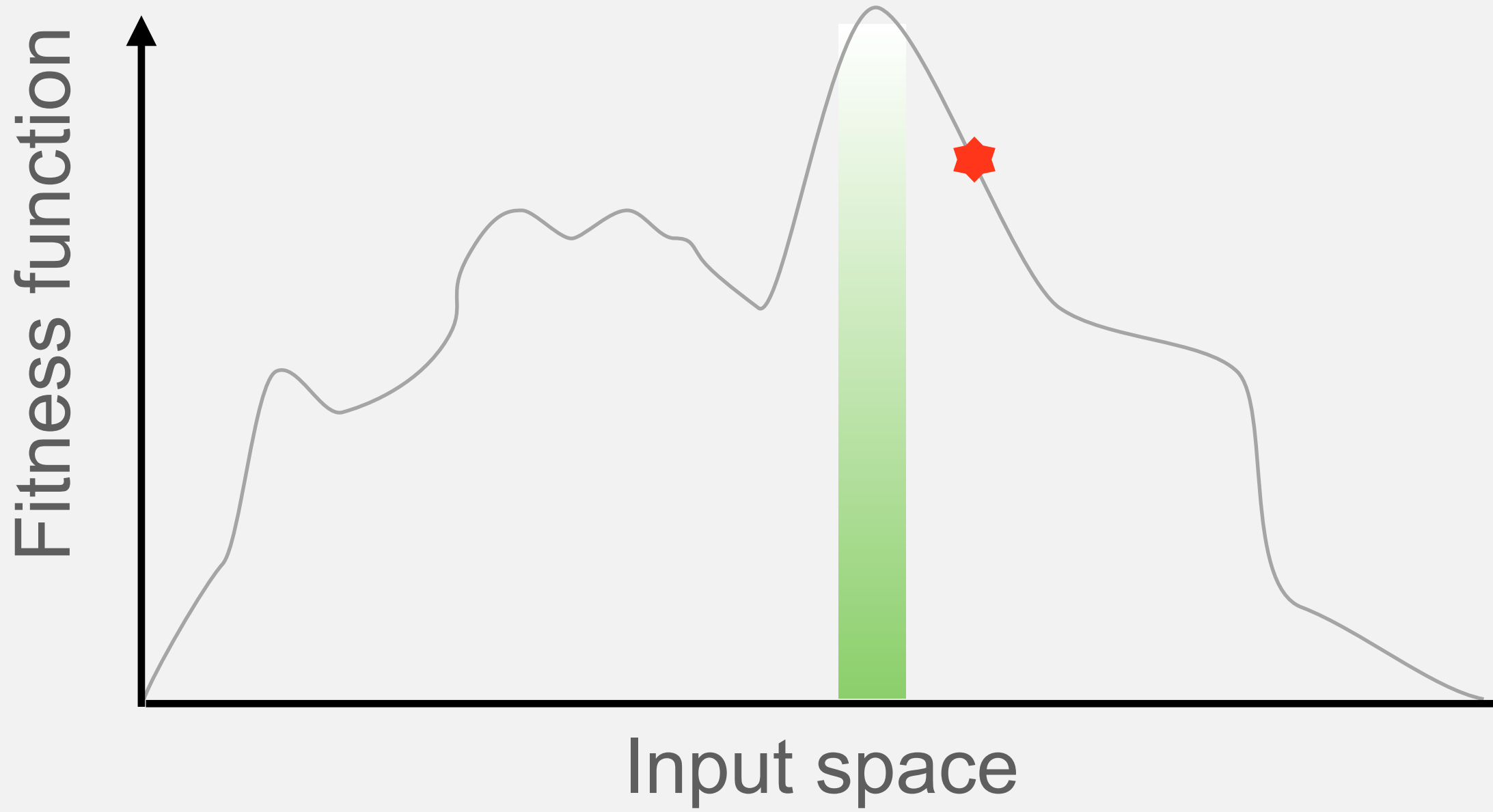
A hand holding a pen over a document with a blue overlay.

Search-Based Testing

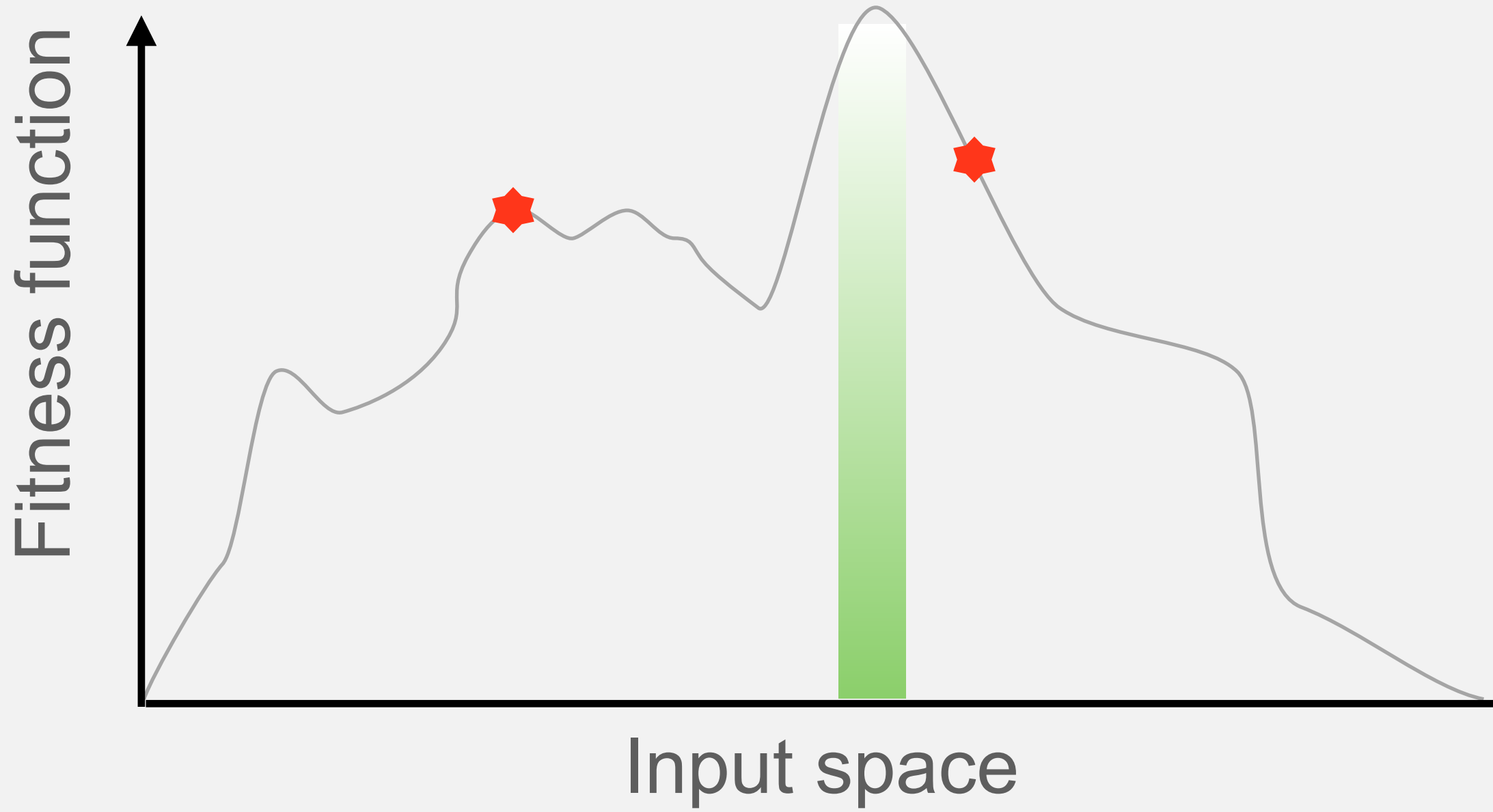
Encoding test coverage as a fitness function



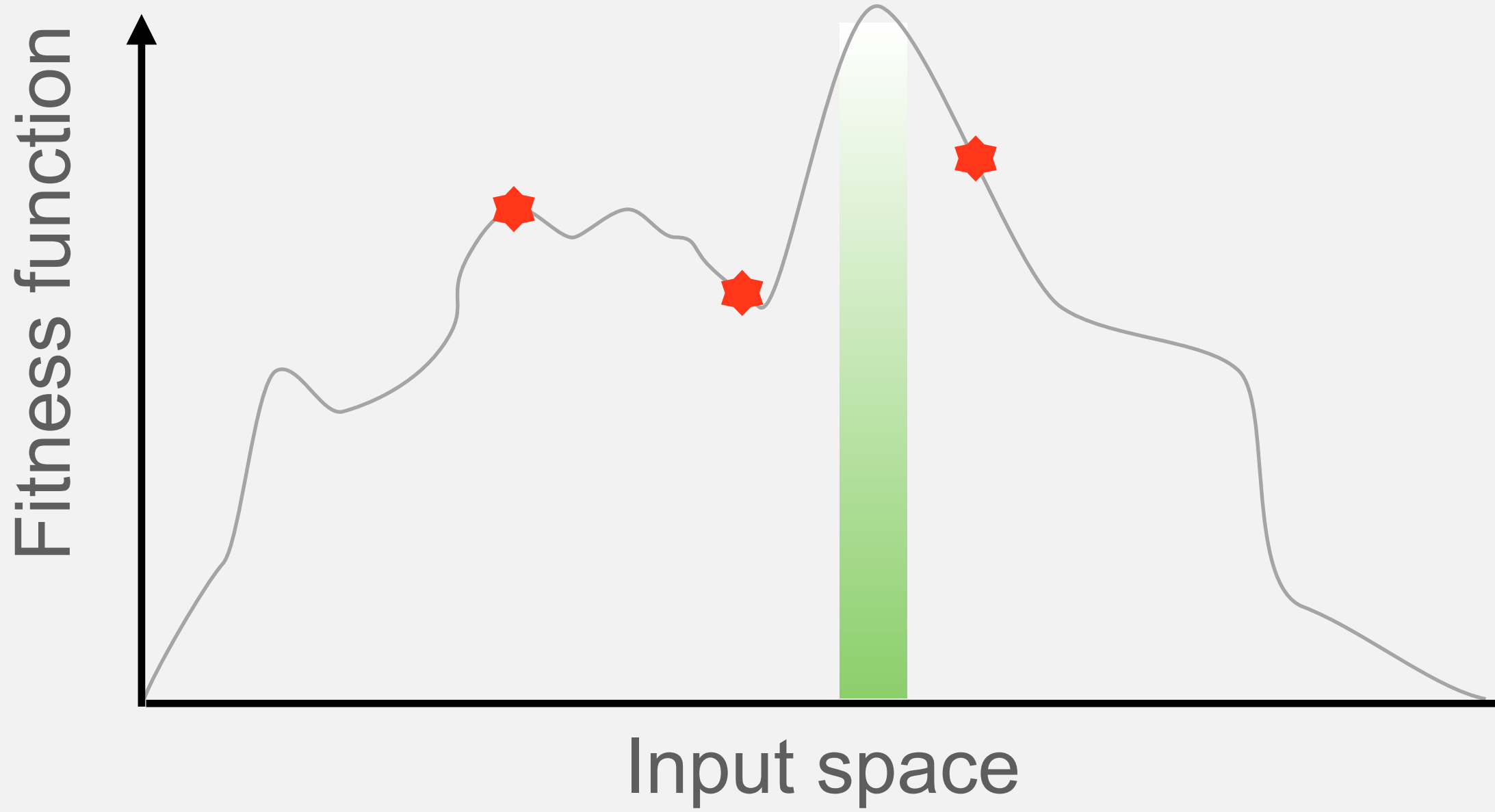
Encoding test coverage as a fitness function



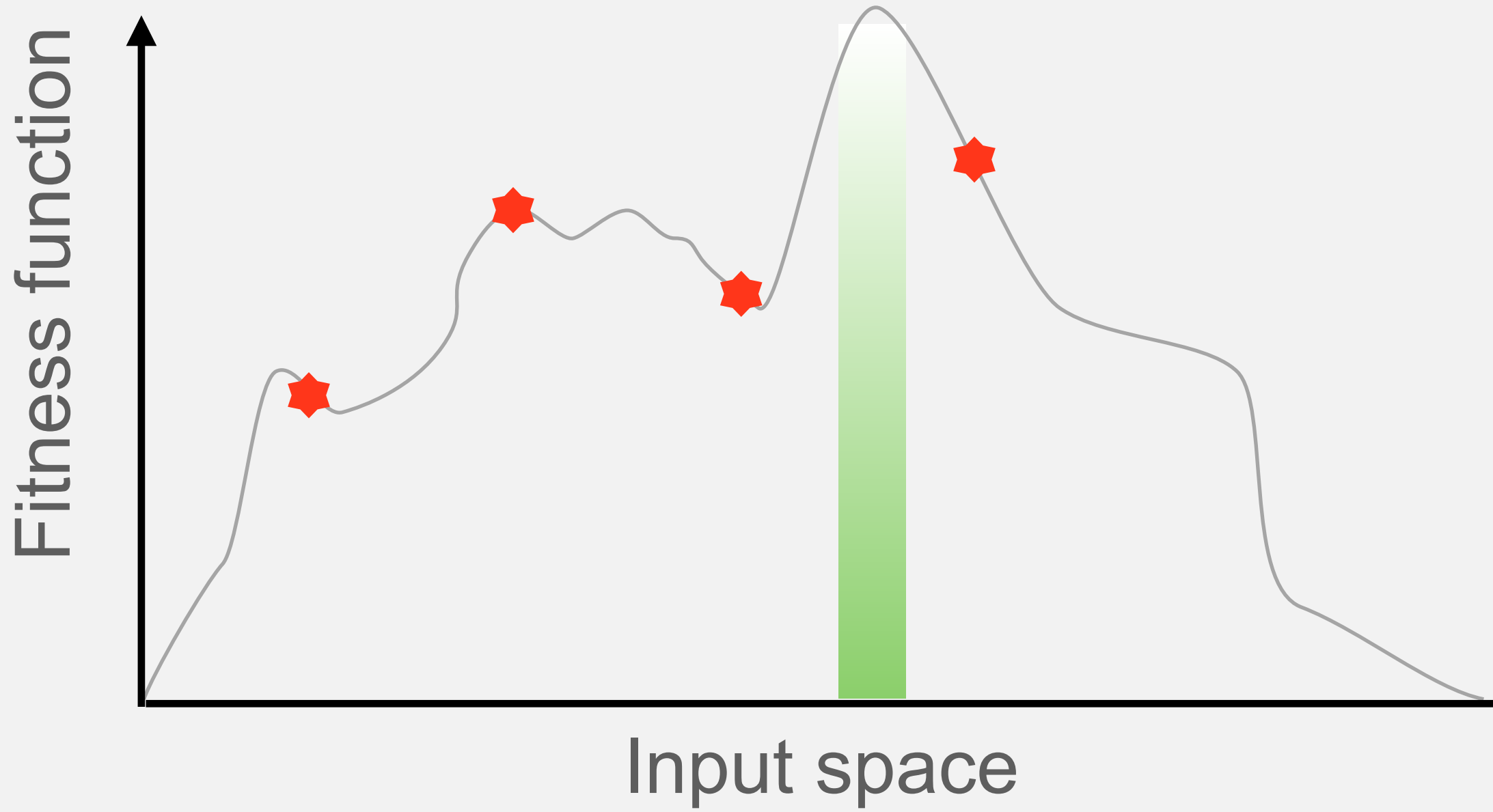
Encoding test coverage as a fitness function



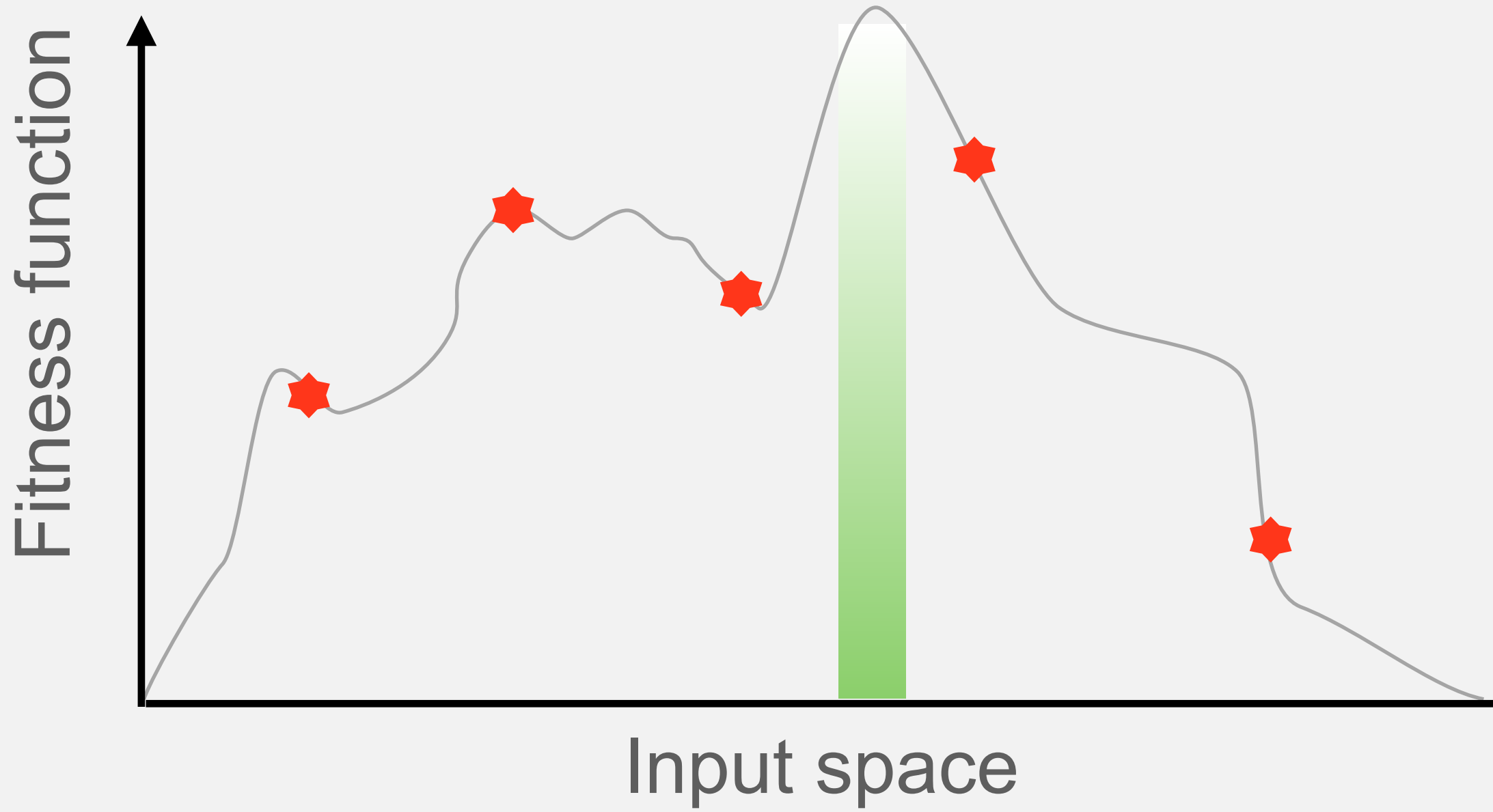
Encoding test coverage as a fitness function



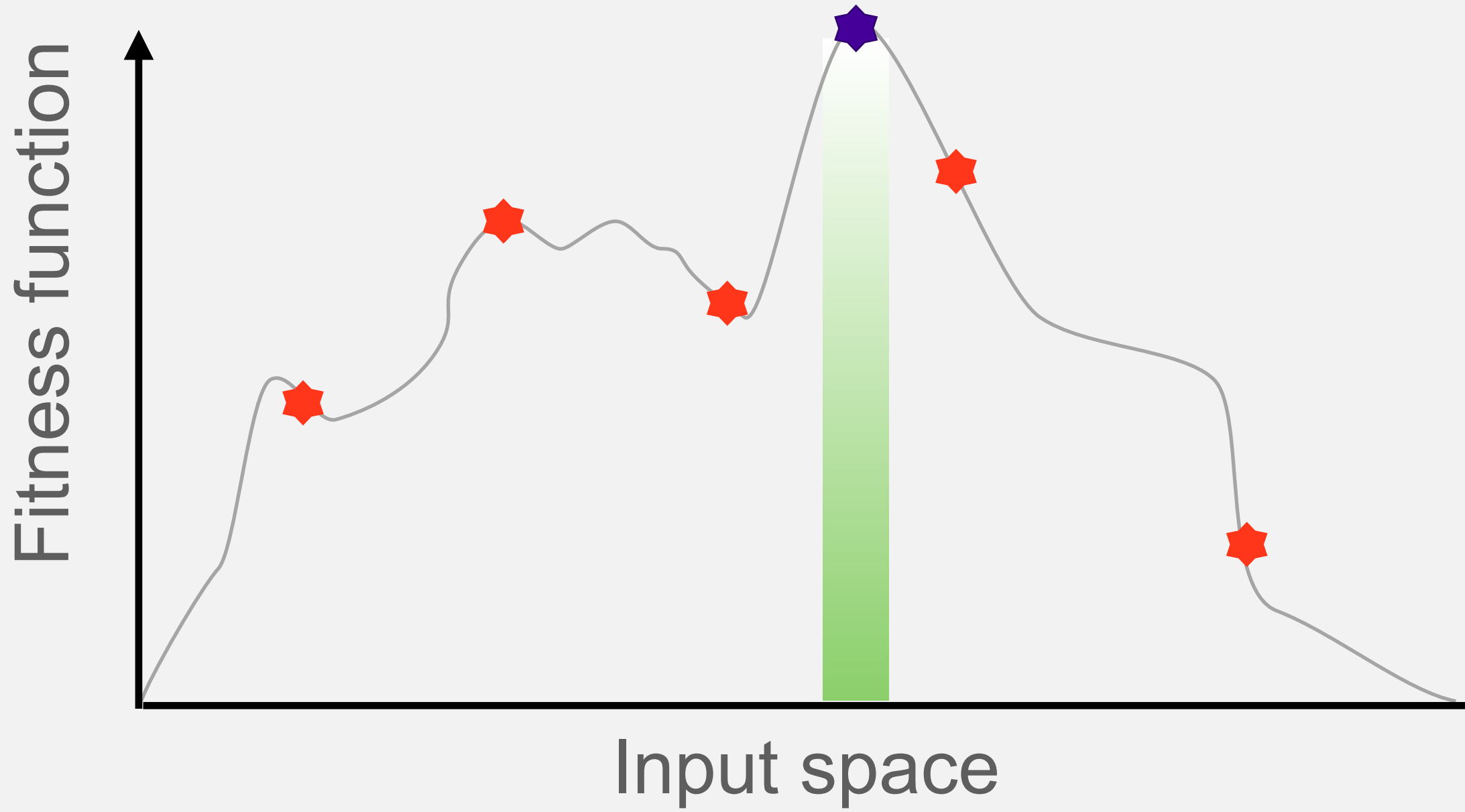
Encoding test coverage as a fitness function



Encoding test coverage as a fitness function



Encoding test coverage as a fitness function



Component of a SBST Tool

Component of a SBST Tool

Search Algorithm

What set of rules or heuristics do we use to process a search-space?

Component of a SBST Tool

Search Algorithm

What set of rules or heuristics do we use to process a search-space?

Representation

How do we represent the search-space so it can be processed by the search algorithm?

Component of a SBST Tool

Search Algorithm

What set of rules or heuristics do we use to process a search-space?

Representation

How do we represent the search-space so it can be processed by the search algorithm?

Search Operators

How do we navigate the search-space (the representation)?

Component of a SBST Tool

Search Algorithm

What set of rules or heuristics do we use to process a search-space?

Representation

How do we represent the search-space so it can be processed by the search algorithm?

Search Operators

How do we navigate the search-space (the representation)?

Fitness Function

How do we evaluate how “good” different test inputs (or test sets) are?

Component of a SBST Tool

Search Algorithm

What set of rules or heuristics do we use to process a search-space?

Representation

How do we represent the search-space so it can be processed by the search algorithm?

Search Operators

How do we navigate the search-space (the representation)?

Fitness Function

How do we evaluate how “good” different test inputs (or test sets) are?

Test Execution

How do we invoke the target system with the proposed inputs?

Component of a SBST Tool

Search Algorithm

What set of rules or heuristics do we use to process a search-space?

Representation

How do we represent the search-space so it can be processed by the search algorithm?

Search Operators

How do we navigate the search-space (the representation)?

Fitness Function

How do we evaluate how “good” different test inputs (or test sets) are?

Test Execution

How do we invoke the target system with the proposed inputs?

Instrumentation

How do we obtain the feedback that we require for our fitness function?

Component of a SBST Tool

Search Algorithm

What set of rules or heuristics do we use to process a search-space?

Representation

How do we represent the search-space so it can be processed by the search algorithm?

Search Operators

How do we navigate the search-space (the representation)?

Fitness Function

How do we evaluate how “good” different test inputs (or test sets) are?

Test Execution

How do we invoke the target system with the proposed inputs?

Instrumentation

How do we obtain the feedback that we require for our fitness function?

Fitness Function: Branch Distance

Fitness function should be zero if we cover our target

Branch Distance measures how close a test case is to covering a specific branch.

Converts a true/false condition into a numerical fitness function.

Often combined with (added to) **Approach Level**.

Number of conditional nodes required to reach target that are not covered.

Example: $\text{if}(x > y) \dots$

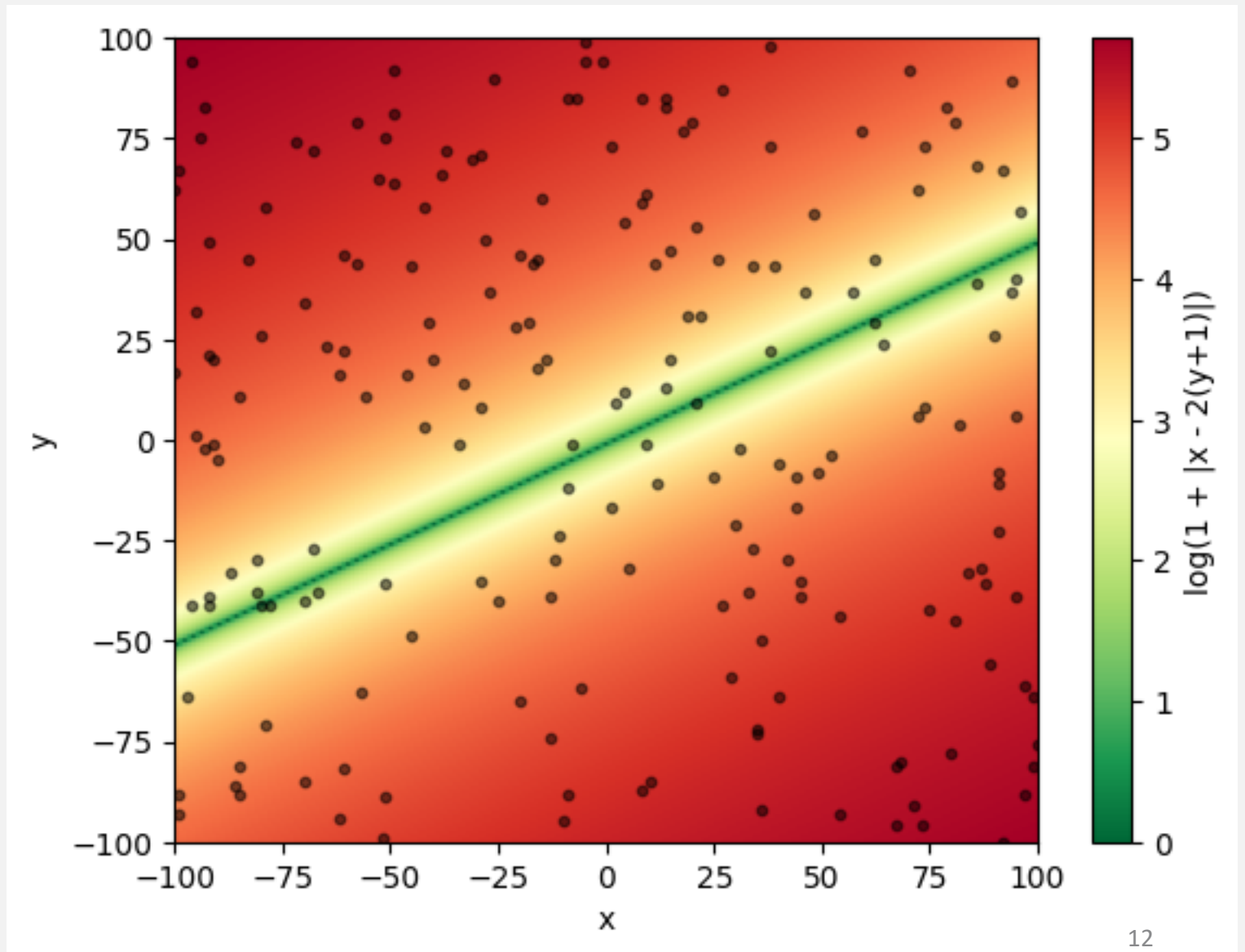
If the branch evaluates to true, $d = 0$.

Otherwise, $d = |x - y|$

Provides search algorithm with a signal.

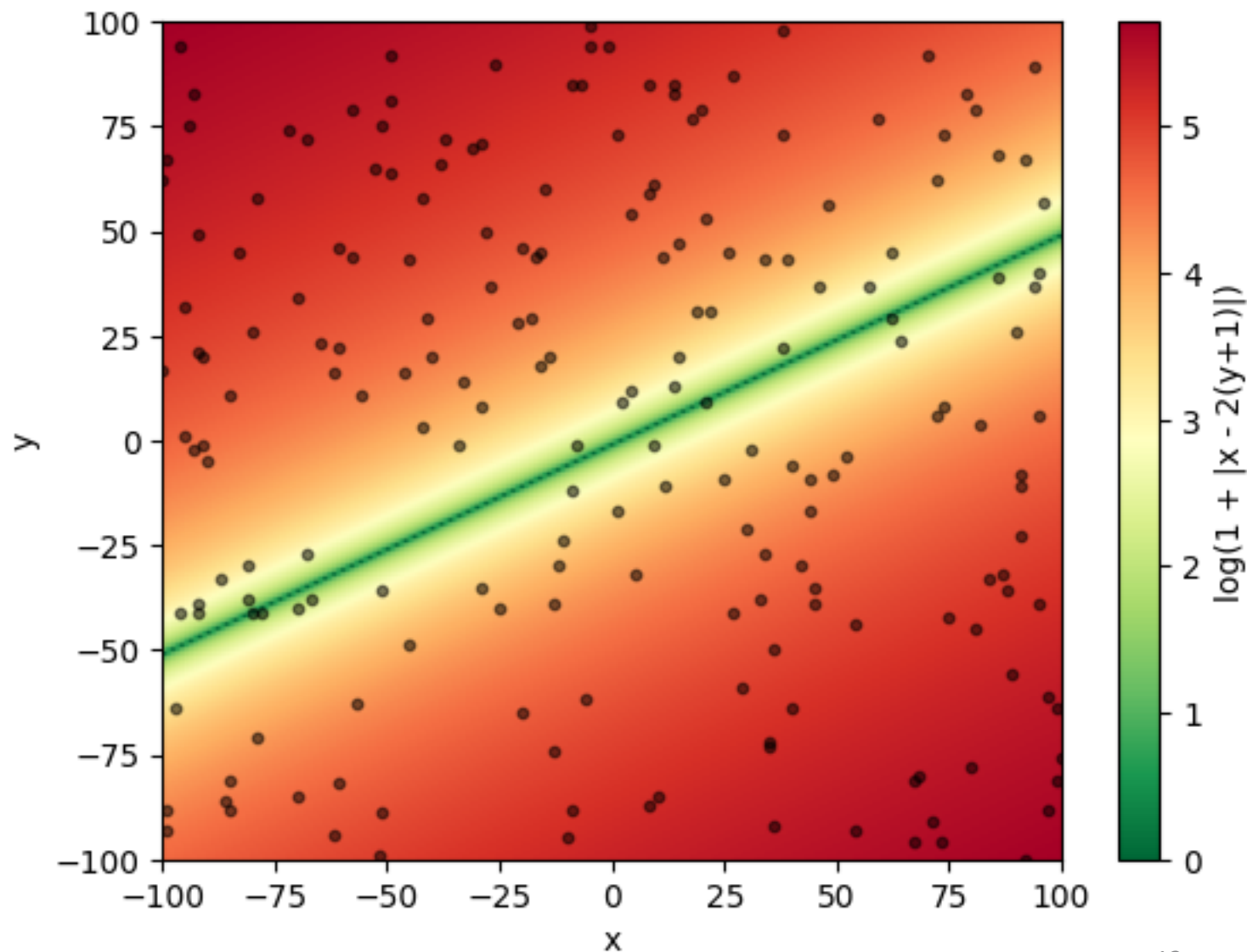
Previous tests can be evaluated and used to find better tests that further reduce the fitness function.

Branch distance for previous example



Branch distance for previous example

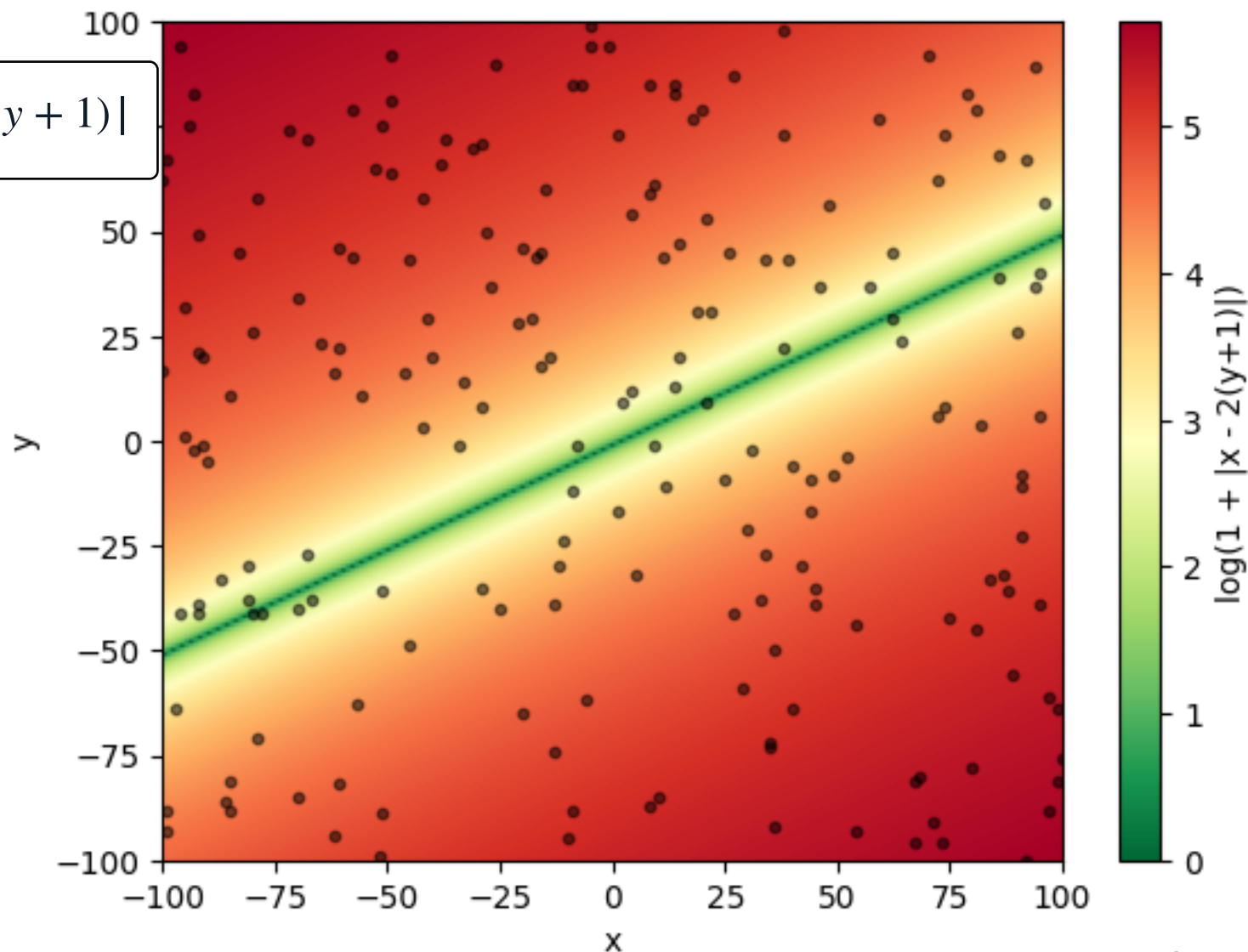
```
def testMe(x, y):  
    if x == 2 * (y + 1):  
        return True  
    else:  
        return False
```



Branch distance for previous example

```
def testMe(x, y):  
    if x == 2 * (y + 1):  
        return True  
    else:  
        return False
```

$$d = |x - 2(y + 1)|$$

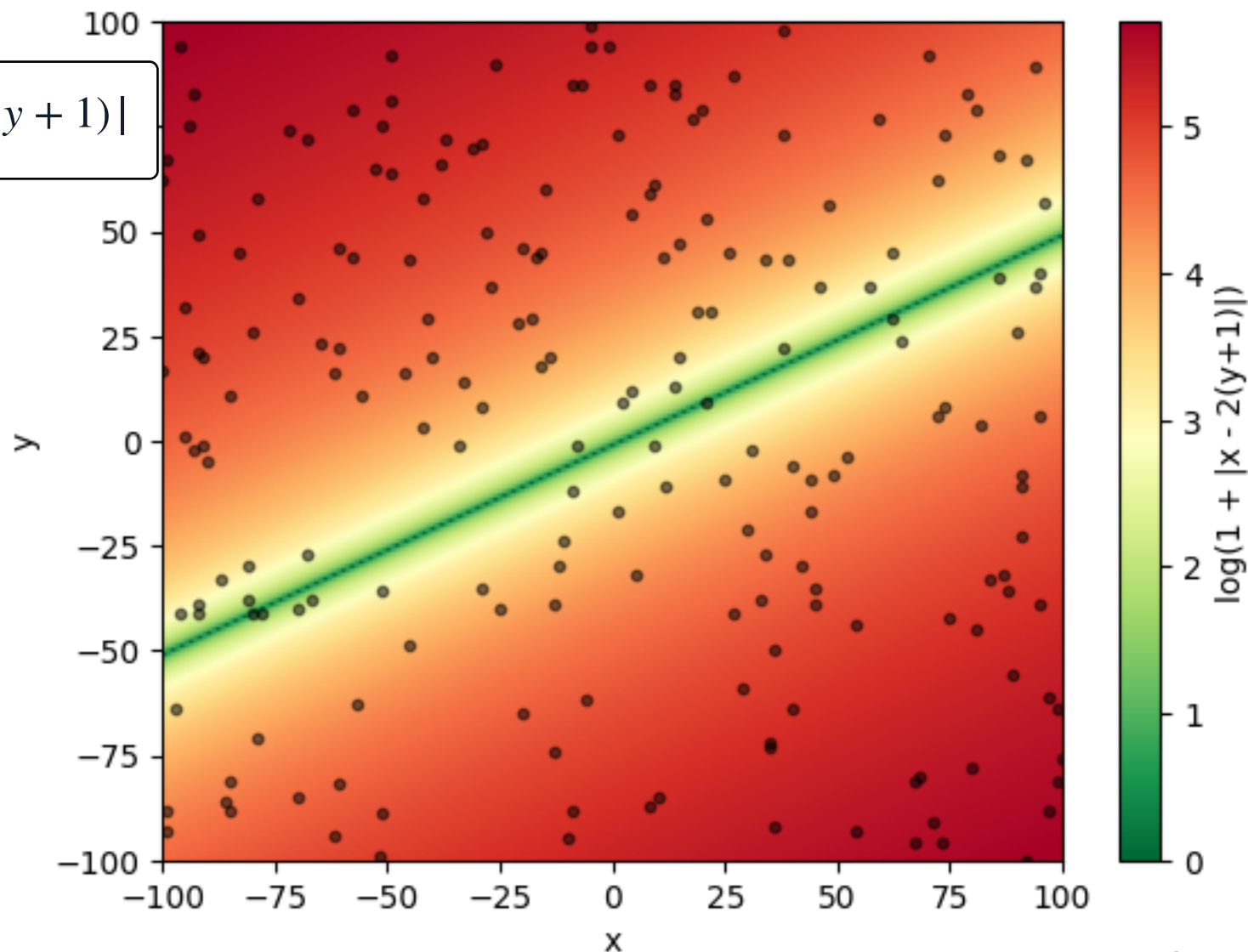


Branch distance for previous example

```
def testMe(x, y):  
    if x == 2 * (y + 1):  
        return True  
    else:  
        return False
```

$$d = |x - 2(y + 1)|$$

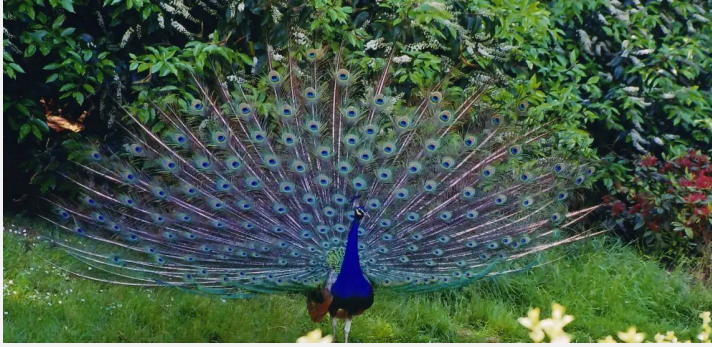
Approach level is trivially zero here.
There are no preceding and necessary
uncovered conditional statements.



A hand holding a pen over a document with mathematical formulas, overlaid with a blue gradient.

Genetic Algorithms for Search-Based Testing

Inspired by Darwin's principle of natural selection



Essential principles

Genes

Specific elements that can affect (often in combination of other genes) the traits of an organism.

Chromosomes

The specific sequence of genes that make up an individual.

Evolution happens when two individuals reproduce.

Their chromosomes are combined (by a process known as crossover).

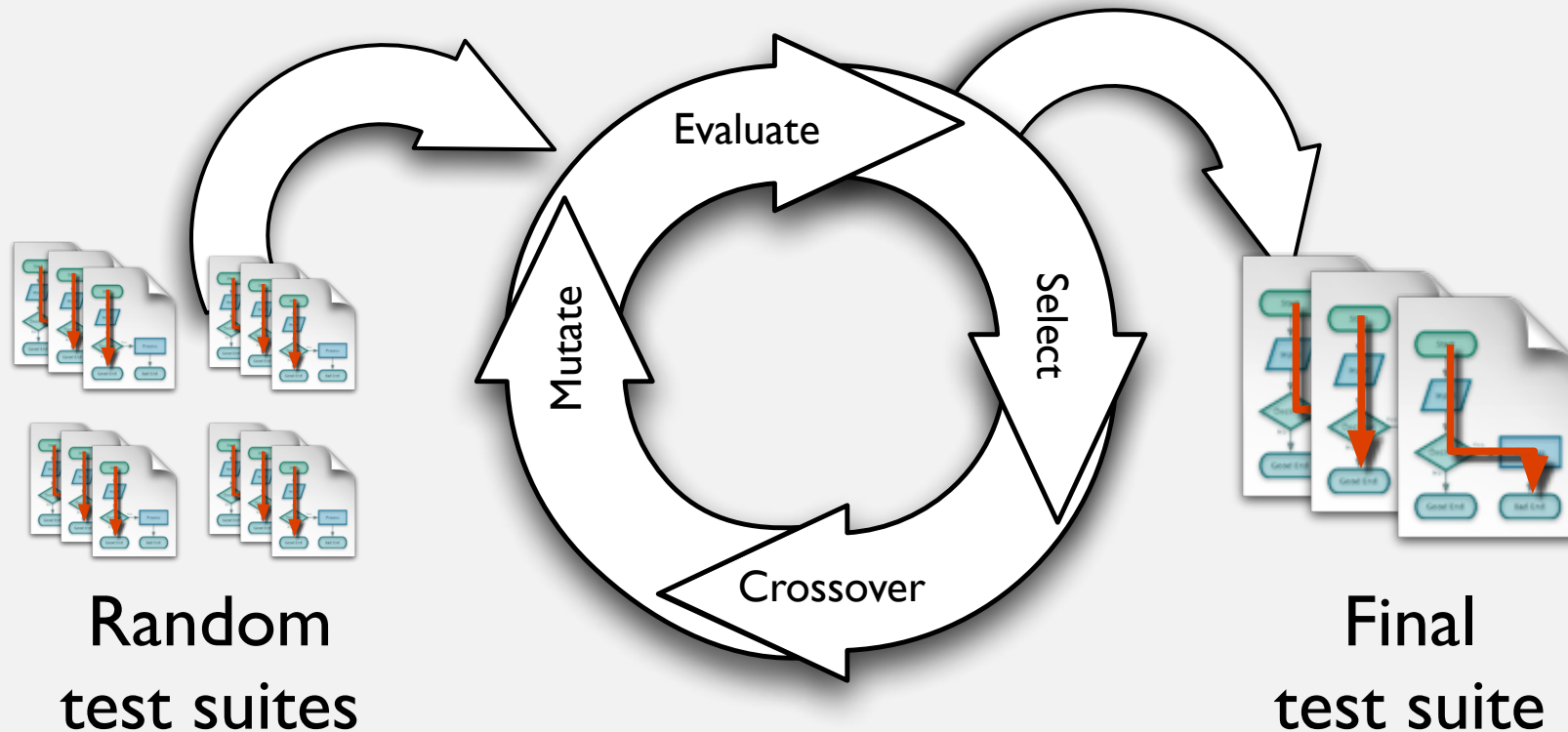
Chromosomes in the offspring can be affected by mutation.

Selection

The “fittest” survive and are selected to reproduce.

The most popular approach to Search-Based Testing

GA search-based testing approaches build upon the fitness function shown previously. Provide a coherent framework of **search algorithm**, **search space representation** and **search operators**.



Components of GA-Based Search-Based Testing

Genes are individual test cases.

Chromosomes are entire test-suites.

The fitness of a chromosome (test-suite) is by, e.g., the sum of branch-distances.

Search operators are cross-over and mutation.

- Crossover, combining the test-cases of predecessors into a new test suite.

- Mutation, changing test inputs for particular genes, adding in new test cases, removing others.

Tests as Individuals

An individual in the search is a test *suite*, i.e., a collection of unit tests:

```
@Test  
  
public void test()  
{  
  
    // set up  
  
    // execute code  
  
    // assertions  
}
```

Tests as Individuals

At first, each test contains a randomly generated sequence of statements

@Test

public void test()

{

int var0 = 10

YearMonthDay var1 = new YearMonthDay(var0);

TimeOfDay var2 = new TimeOfDay();

DateTime var3 = var1.toDateTime(var2);

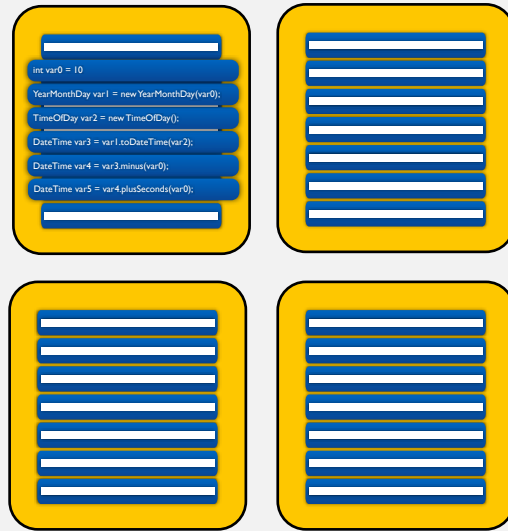
DateTime var4 = var3.minus(var0);

DateTime var5 = var4.plusSeconds(var0);

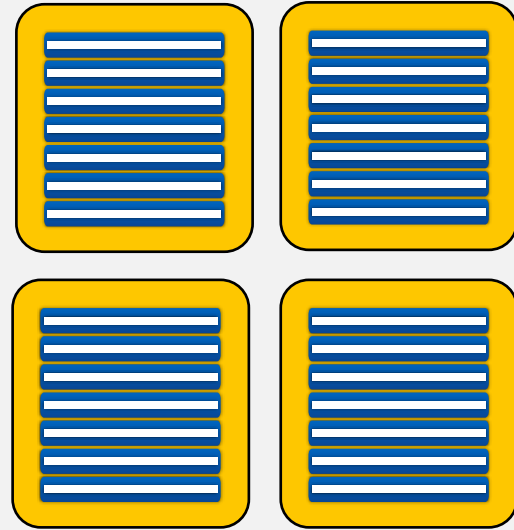
}

Assertions are not generated at this stage,
they are added as a post process¹⁹

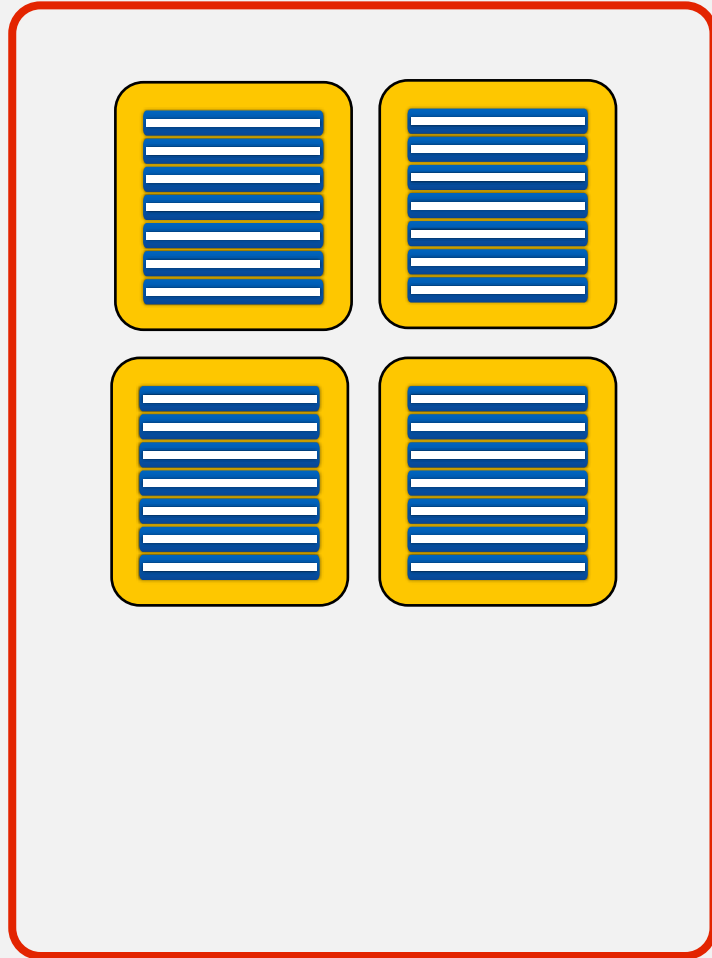
Test Suite generation



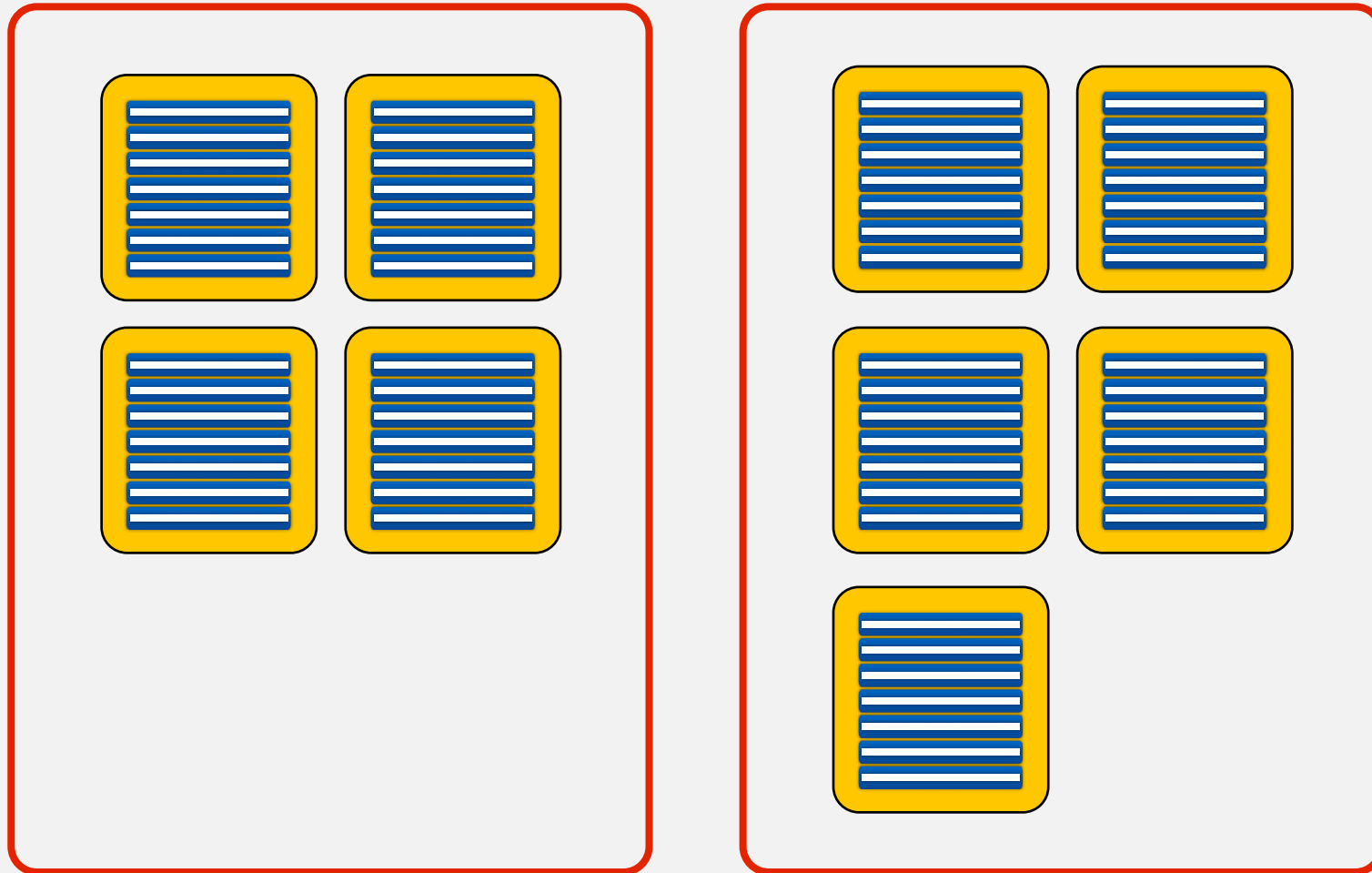
Test Suite generation



Test Suite generation

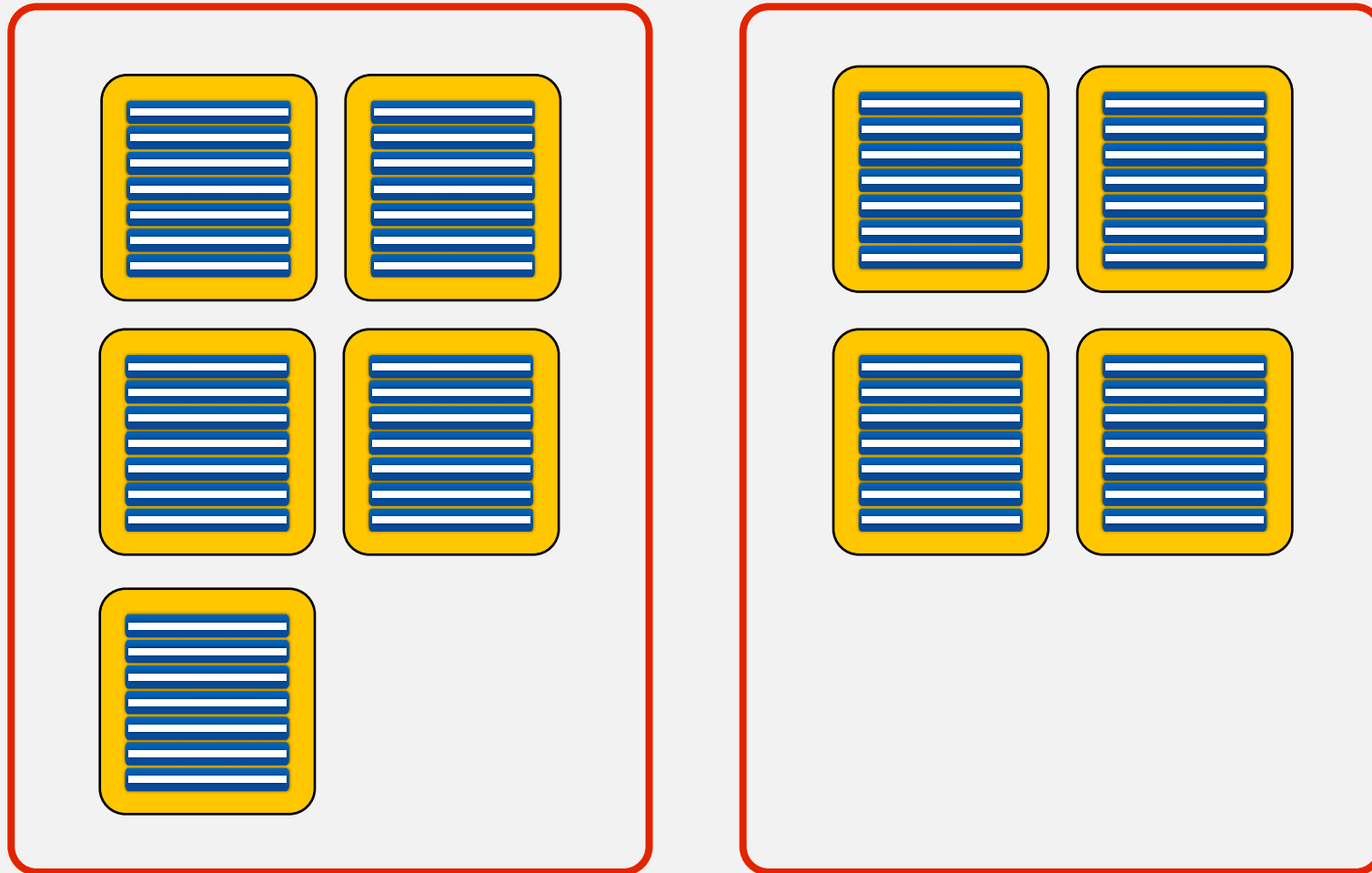


Test Suite generation



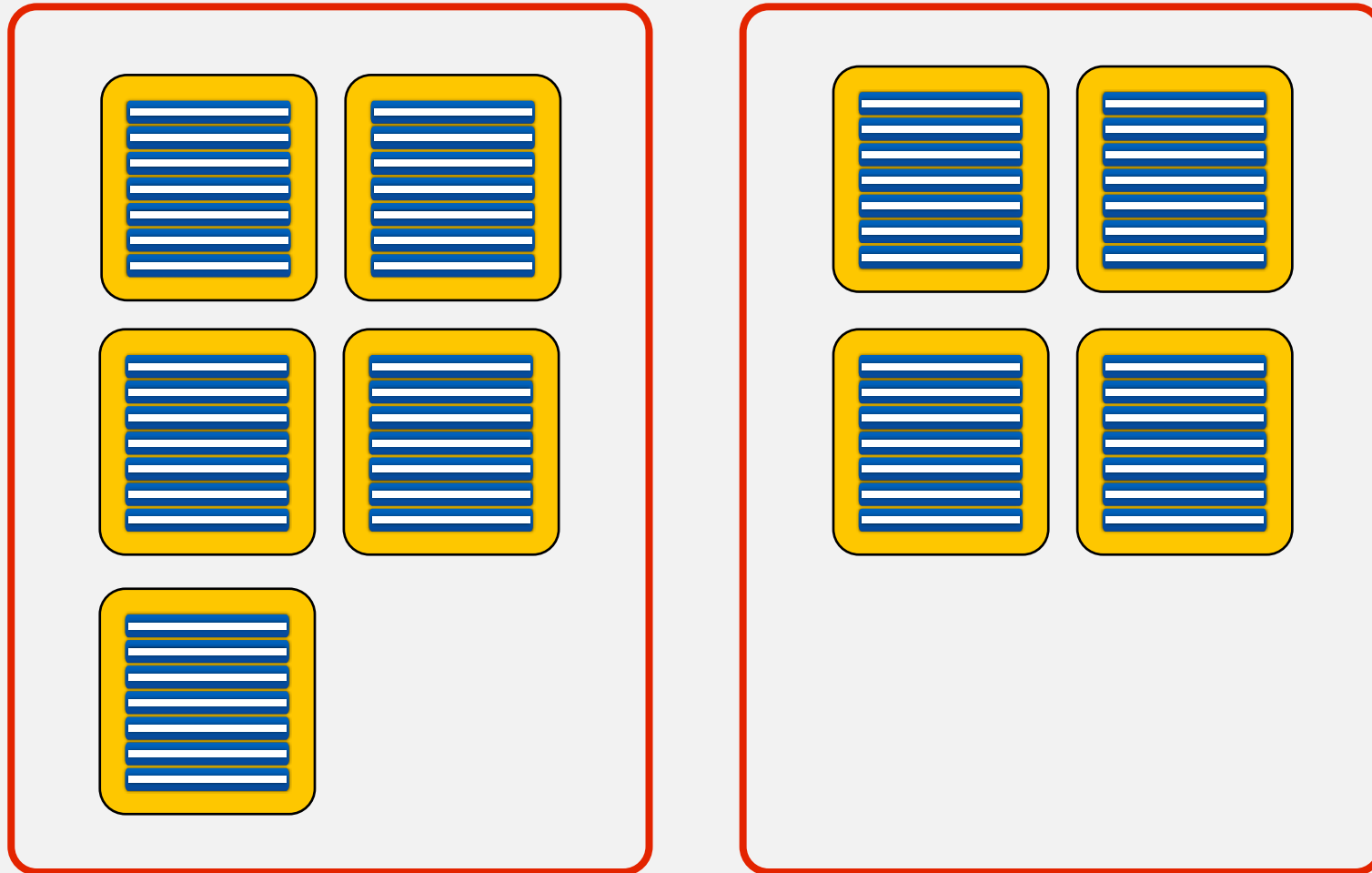
The size of the initial population is configurable

Crossover



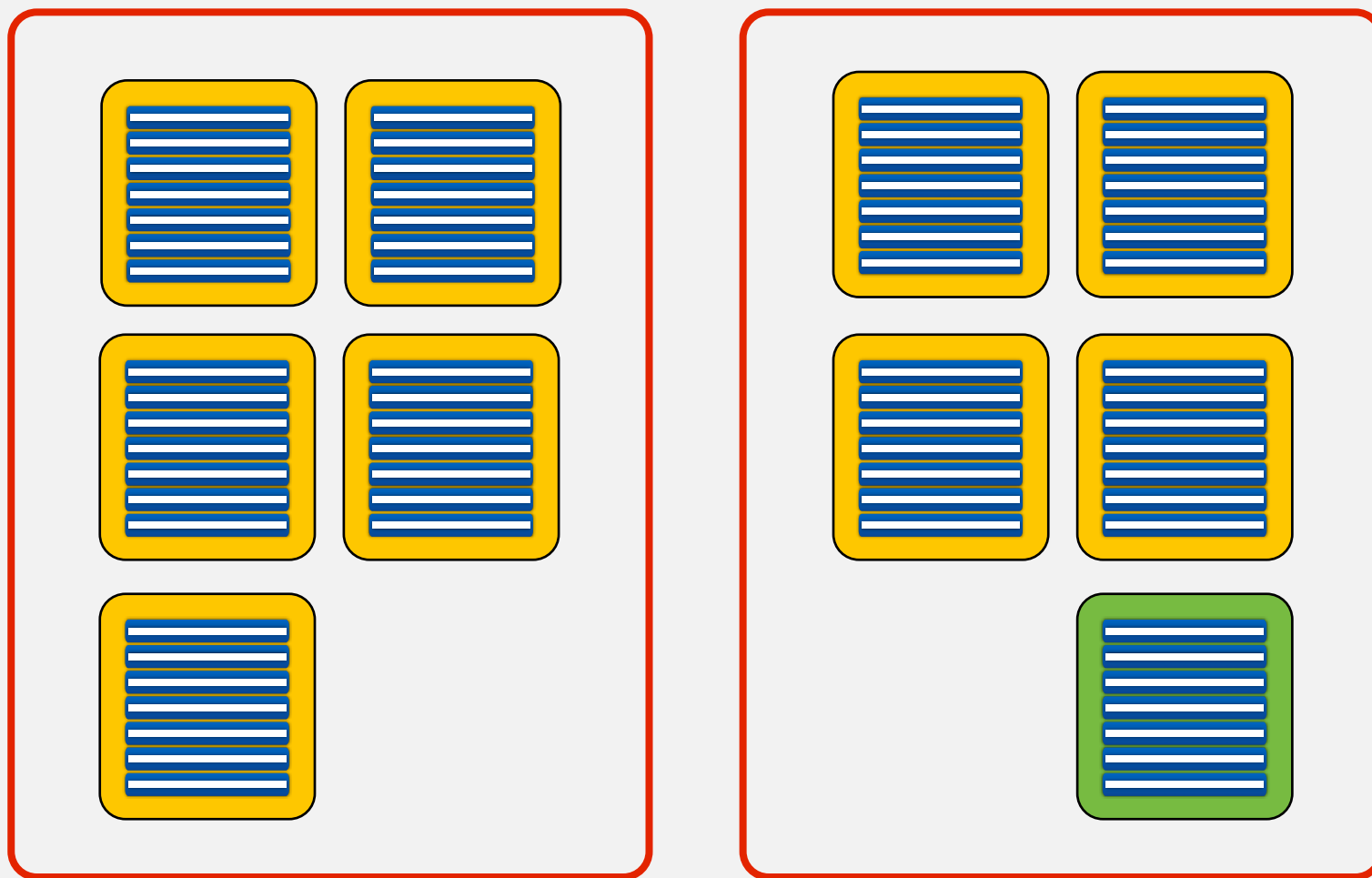
Crossover happens with a certain probability

Mutation



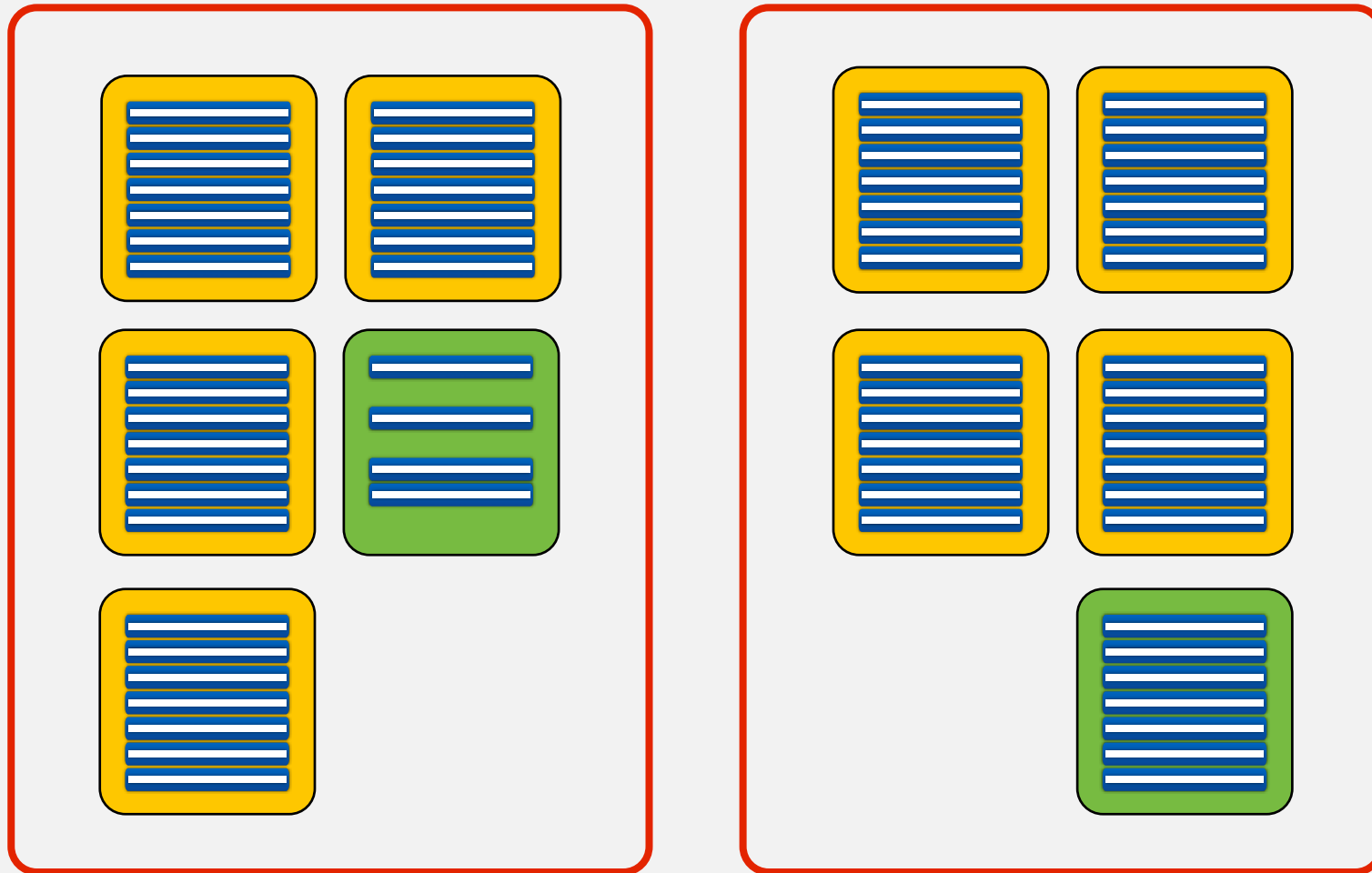
Mutation happens with a certain probability

Mutation



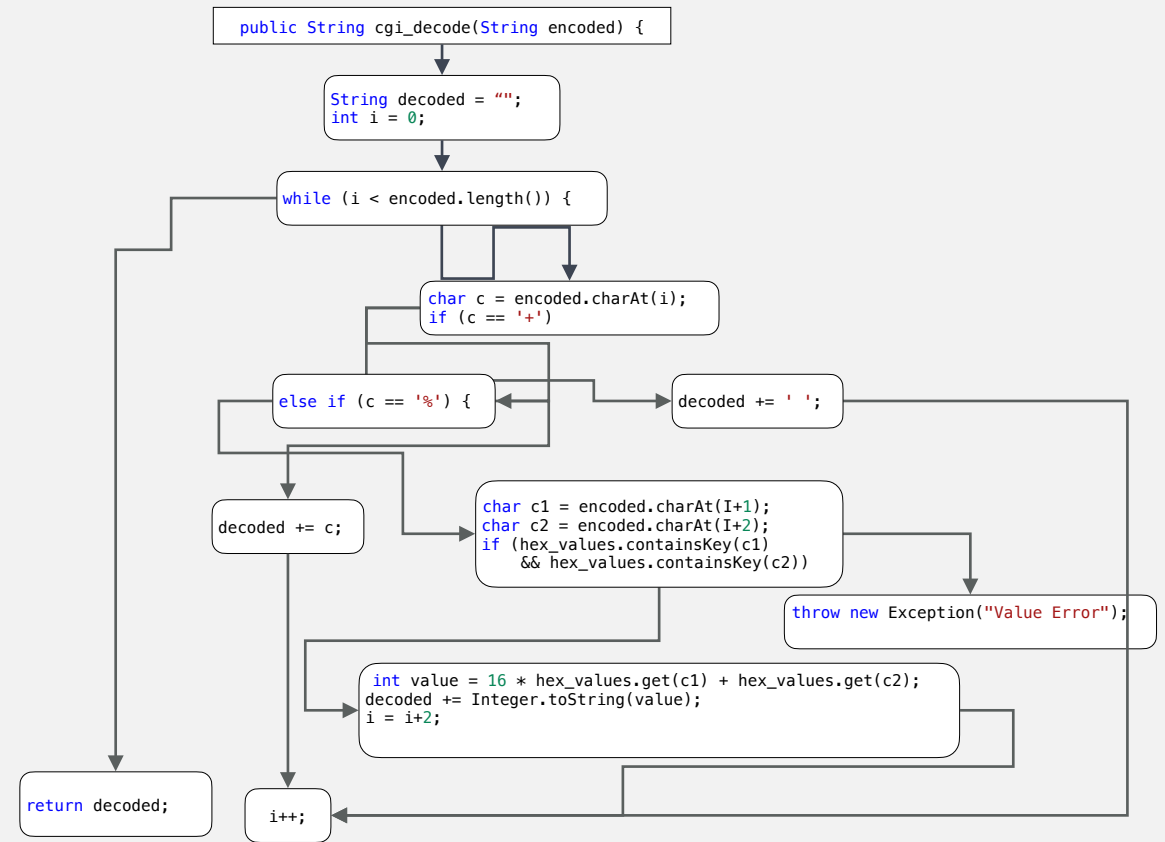
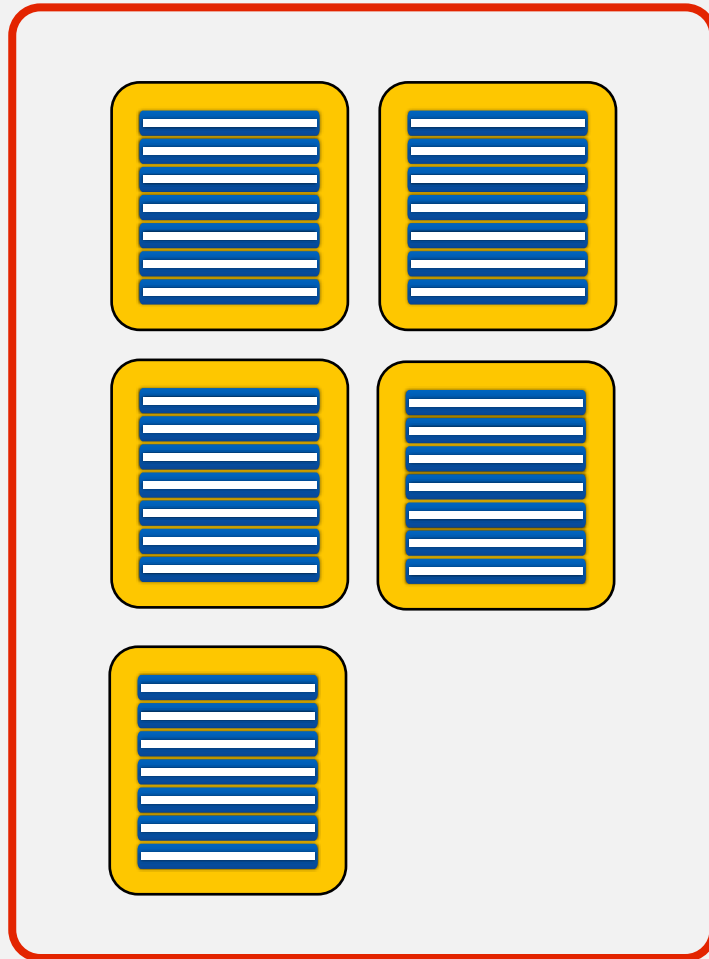
Mutation happens with a certain probability

Mutation



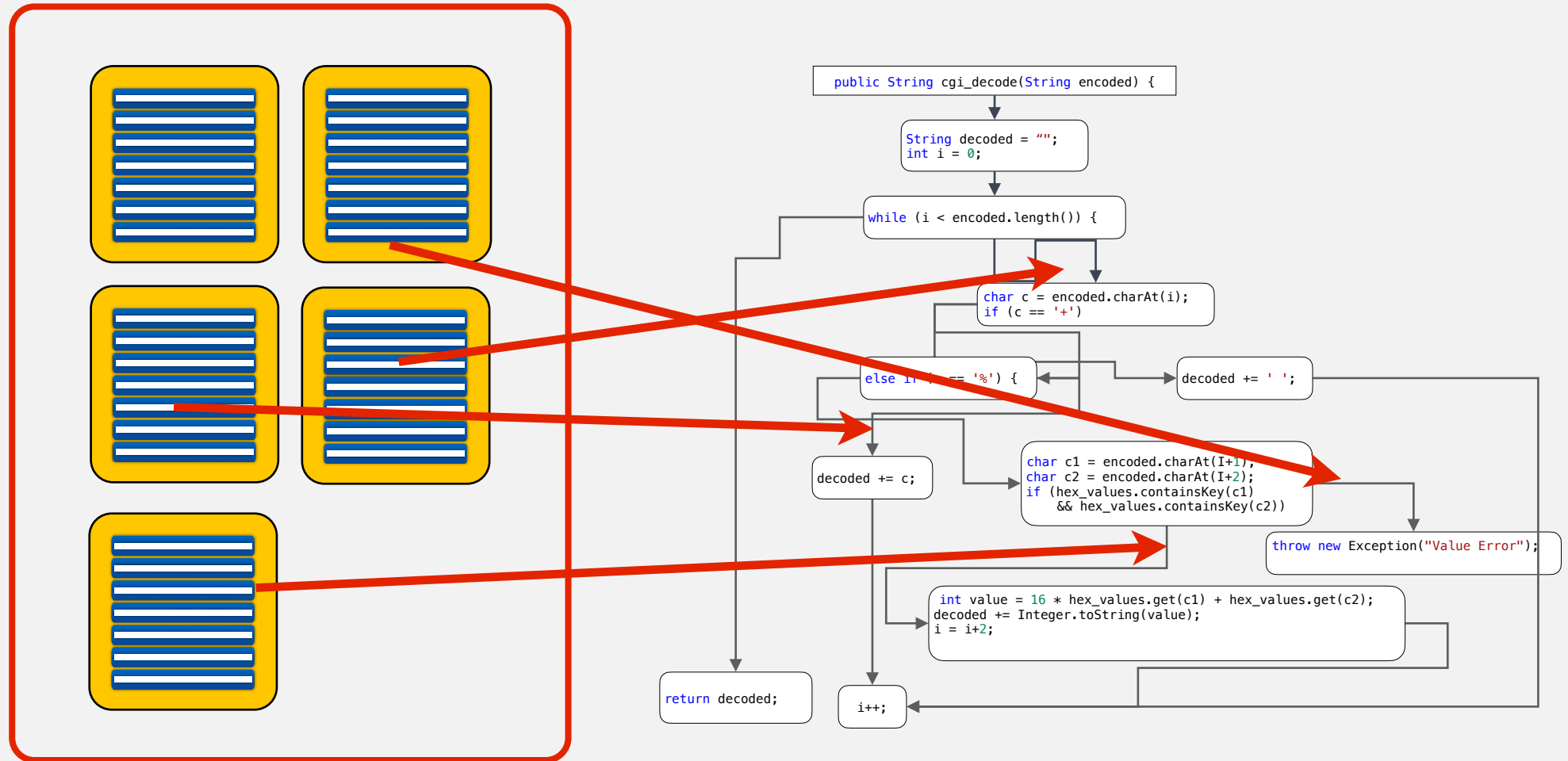
Mutation happens with a certain probability

Fitness evaluation



Fitness is computed using *branch distance* and *approach level*

Fitness evaluation



Fitness is computed using *branch distance* and *approach level*

A background image showing a hand holding a pen and writing on a notepad. The image is overlaid with a semi-transparent blue filter. The text is centered in the lower half of the image.

Illustration on our toy example

```
import pygad
import numpy as np
import matplotlib.pyplot as plt
```

```
best_x_history = []
best_y_history = []
```

```
def fitness_func(ga_instance, solution, solution_idx):
    x, y = solution
    branch_distance = abs(x - 2 * (y + 1))
    return 1.0 / (branch_distance + 0.0001)
```

```
def on_generation(ga_instance):
    generation = ga_instance.generations_completed
    best_sol, best_fit, _ = ga_instance.best_solution()
    x, y = int(best_sol[0]), int(best_sol[1])

    best_x_history.append(x)
    best_y_history.append(y)

    print(f"Gen {generation:02d} | Best Solution: x={x:>4}, y={y:>4} | Branch
Distance: {abs(x - 2 * (y + 1))}")
```

```
print("Running Genetic Algorithm...\n")
```

```
ga_instance = pygad.GA(
    num_generations=100,
    sol_per_pop=40,
    num_parents_mating=20,
    fitness_func=fitness_func,
    num_genes=2,
    gene_type=int,
    init_range_low=-5000,
    init_range_high=5000,
    crossover_type="single_point",
    mutation_type="random",
    mutation_num_genes=1,
    random_mutation_min_val=-5,
    random_mutation_max_val=5,
    mutation_by_replacement=False,
    on_generation=on_generation
)
```

```
ga_instance.run()
```

Gen 01	Best Solution: x=4143	y=2239	Branch Distance: 337
Gen 02	Best Solution: x=1079	y= 434	Branch Distance: 209
Gen 03	Best Solution: x=1079	y= 434	Branch Distance: 209
Gen 04	Best Solution: x=1078	y= 434	Branch Distance: 208
Gen 05	Best Solution: x=1074	y= 434	Branch Distance: 204
Gen 06	Best Solution: x=1072	y= 434	Branch Distance: 202
Gen 07	Best Solution: x=1072	y= 437	Branch Distance: 196
Gen 08	Best Solution: x=1067	y= 438	Branch Distance: 189
Gen 09	Best Solution: x=1071	y= 441	Branch Distance: 187
Gen 10	Best Solution: x=1071	y= 445	Branch Distance: 179
Gen 11	Best Solution: x=1069	y= 445	Branch Distance: 177
Gen 12	Best Solution: x=1069	y= 445	Branch Distance: 177
Gen 13	Best Solution: x=1066	y= 444	Branch Distance: 176
Gen 14	Best Solution: x=1063	y= 445	Branch Distance: 171
Gen 15	Best Solution: x=1057	y= 446	Branch Distance: 163
Gen 16	Best Solution: x=1058	y= 448	Branch Distance: 160
Gen 17	Best Solution: x=1057	y= 449	Branch Distance: 157
Gen 18	Best Solution: x=1058	y= 456	Branch Distance: 144
Gen 19	Best Solution: x=1058	y= 456	Branch Distance: 144
Gen 20	Best Solution: x=1048	y= 455	Branch Distance: 136
Gen 21	Best Solution: x=1044	y= 459	Branch Distance: 124
Gen 22	Best Solution: x=1041	y= 460	Branch Distance: 119
Gen 23	Best Solution: x=1037	y= 460	Branch Distance: 115
Gen 24	Best Solution: x=1035	y= 460	Branch Distance: 113
Gen 25	Best Solution: x=1045	y= 467	Branch Distance: 109
Gen 26	Best Solution: x=1043	y= 467	Branch Distance: 107
Gen 27	Best Solution: x=1029	y= 469	Branch Distance: 89
Gen 28	Best Solution: x=1029	y= 469	Branch Distance: 89
Gen 29	Best Solution: x=1029	y= 471	Branch Distance: 85
Gen 30	Best Solution: x=1024	y= 470	Branch Distance: 82
Gen 31	Best Solution: x=1024	y= 473	Branch Distance: 76
Gen 32	Best Solution: x=1024	y= 475	Branch Distance: 72
Gen 33	Best Solution: x=1018	y= 476	Branch Distance: 64
Gen 34	Best Solution: x=1016	y= 477	Branch Distance: 60
Gen 35	Best Solution: x=1011	y= 477	Branch Distance: 55
Gen 36	Best Solution: x=1011	y= 479	Branch Distance: 51
Gen 37	Best Solution: x=1008	y= 480	Branch Distance: 46

```
import pygad
import numpy as np
import matplotlib.pyplot as plt
```

```
best_x_history = []
best_y_history = []
```

```
def fitness_func(ga_instance, solution, solution_idx):
    x, y = solution
    branch_distance = abs(x - 2 * (y + 1))
    return 1.0 / (branch_distance + 0.0001)
```

```
def on_generation(ga_instance):
    generation = ga_instance.generations_completed
    best_sol, best_fit, _ = ga_instance.best_solution()
    x, y = int(best_sol[0]), int(best_sol[1])

    best_x_history.append(x)
    best_y_history.append(y)

    print(f"Gen {generation:02d} | Best Solution: x={x:>4}, y={y:>4} | Branch
Distance: {abs(x - 2 * (y + 1))}")
```

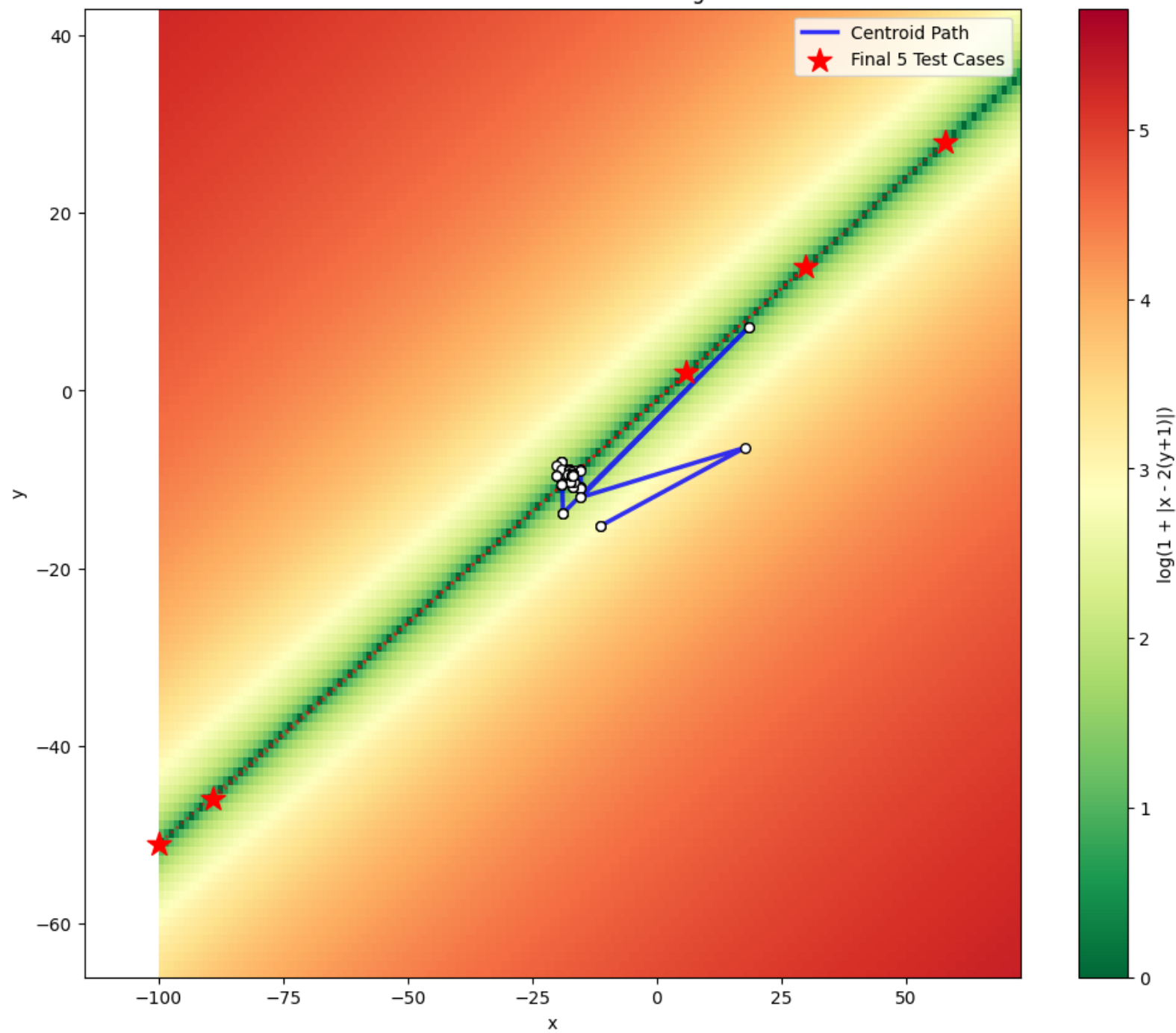
```
print("Running Genetic Algorithm...\n")
```

```
ga_instance = pygad.GA(
    num_generations=100,
    sol_per_pop=40,
    num_parents_mating=20,
    fitness_func=fitness_func,
    num_genes=2,
    gene_type=int,
    init_range_low=-5000,
    init_range_high=5000,
    crossover_type="single_point",
    mutation_type="random",
    mutation_num_genes=1,
    random_mutation_min_val=-5,
    random_mutation_max_val=5,
    mutation_by_replacement=False,
    on_generation=on_generation
)
```

```
ga_instance.run()
```

Gen 14		Best Solution: x=1063	y= 445		Branch Distance: 171
Gen 15		Best Solution: x=1057	y= 446		Branch Distance: 163
Gen 16		Best Solution: x=1058	y= 448		Branch Distance: 160
Gen 17		Best Solution: x=1057	y= 449		Branch Distance: 157
Gen 18		Best Solution: x=1058	y= 456		Branch Distance: 144
Gen 19		Best Solution: x=1058	y= 456		Branch Distance: 144
Gen 20		Best Solution: x=1048	y= 455		Branch Distance: 136
Gen 21		Best Solution: x=1044	y= 459		Branch Distance: 124
Gen 22		Best Solution: x=1041	y= 460		Branch Distance: 119
Gen 23		Best Solution: x=1037	y= 460		Branch Distance: 115
Gen 24		Best Solution: x=1035	y= 460		Branch Distance: 113
Gen 25		Best Solution: x=1045	y= 467		Branch Distance: 109
Gen 26		Best Solution: x=1043	y= 467		Branch Distance: 107
Gen 27		Best Solution: x=1029	y= 469		Branch Distance: 89
Gen 28		Best Solution: x=1029	y= 469		Branch Distance: 89
Gen 29		Best Solution: x=1029	y= 471		Branch Distance: 85
Gen 30		Best Solution: x=1024	y= 470		Branch Distance: 82
Gen 31		Best Solution: x=1024	y= 473		Branch Distance: 76
Gen 32		Best Solution: x=1024	y= 475		Branch Distance: 72
Gen 33		Best Solution: x=1018	y= 476		Branch Distance: 64
Gen 34		Best Solution: x=1016	y= 477		Branch Distance: 60
Gen 35		Best Solution: x=1011	y= 477		Branch Distance: 55
Gen 36		Best Solution: x=1011	y= 479		Branch Distance: 51
Gen 37		Best Solution: x=1008	y= 480		Branch Distance: 46
Gen 38		Best Solution: x=1011	y= 483		Branch Distance: 43
Gen 39		Best Solution: x=1009	y= 483		Branch Distance: 41
Gen 40		Best Solution: x=1008	y= 484		Branch Distance: 38
Gen 41		Best Solution: x=1000	y= 484		Branch Distance: 30
Gen 42		Best Solution: x= 995	y= 483		Branch Distance: 27
Gen 43		Best Solution: x= 994	y= 486		Branch Distance: 20
Gen 44		Best Solution: x= 994	y= 493		Branch Distance: 6
Gen 45		Best Solution: x= 987	y= 492		Branch Distance: 1
Gen 46		Best Solution: x= 987	y= 492		Branch Distance: 1
Gen 47		Best Solution: x= 992	y= 495		Branch Distance: 0

Path of the Set's Centroid Towards Target Branch





Strengths and Weaknesses

Strengths

Can automatically generate test sets that optimise for branch coverage.

Or any other form of coverage that can be encoded as a fitness function.

Strengths

Can automatically generate test sets that optimise for branch coverage.

Or any other form of coverage that can be encoded as a fitness function.

Easy to implement.

Up to you to define a fitness function, representation and search space.

Leave the “hard work” to search algorithms.

Lots of established library support for Genetic Algorithms.

Strengths

Can automatically generate test sets that optimise for branch coverage.

Or any other form of coverage that can be encoded as a fitness function.

Easy to implement.

Up to you to define a fitness function, representation and search space.

Leave the “hard work” to search algorithms.

Lots of established library support for Genetic Algorithms.

Lots of tool support

Evosuite for Java (<https://github.com/EvoSuite/evosuite>)

Penguin for Python (<https://github.com/se2p/pynguin>)

Weaknesses

Readability

Generated tests can be hard to follow.

If a test fails, why is it failing?

```
@Test(timeout = 4000)
public void test01() throws Throwable {
    LinkedList<String> linkedList0 = new LinkedList<String>();
    LinkedList<Object> linkedList1 =
        new LinkedList<Object>((Collection<?>) linkedList0);
    FixedOrderComparator fixedOrderComparator0 =
        new FixedOrderComparator((List) linkedList1);
    linkedList1.add((Object) linkedList1);
    // Undeclared exception!
    try {
        fixedOrderComparator0.compare(linkedList1, linkedList0);
        fail("Expecting exception: StackOverflowError");
    } catch (StackOverflowError e) {
        //
        // no message in exception (getMessage() returned null)
        //
    }
}
```

Weaknesses

Readability

Generated tests can be hard to follow.

If a test fails, why is it failing?

Oracles

Can automate test input generation, but no support for generating test oracles.

```
@Test(timeout = 4000)
public void test01() throws Throwable {
    LinkedList<String> linkedList0 = new LinkedList<String>();
    LinkedList<Object> linkedList1 =
        new LinkedList<Object>((Collection<?>) linkedList0);
    FixedOrderComparator fixedOrderComparator0 =
        new FixedOrderComparator((List) linkedList1);
    linkedList1.add((Object) linkedList1);
    // Undeclared exception!
    try {
        fixedOrderComparator0.compare(linkedList1, linkedList0);
        fail("Expecting exception: StackOverflowError");
    } catch (StackOverflowError e) {
        //
        // no message in exception (getMessage() returned null)
        //
    }
}
```

Weaknesses

Readability

Generated tests can be hard to follow.

If a test fails, why is it failing?

Oracles

Can automate test input generation, but no support for generating test oracles.

Finding the right fitness function.

As we know, coverage isn't everything.

Tests are only as good as the fitness function.

```
@Test(timeout = 4000)
public void test01() throws Throwable {
    LinkedList<String> linkedList0 = new LinkedList<String>();
    LinkedList<Object> linkedList1 =
        new LinkedList<Object>((Collection<?>) linkedList0);
    FixedOrderComparator fixedOrderComparator0 =
        new FixedOrderComparator((List) linkedList1);
    linkedList1.add((Object) linkedList1);
    // Undeclared exception!
    try {
        fixedOrderComparator0.compare(linkedList1, linkedList0);
        fail("Expecting exception: StackOverflowError");
    } catch (StackOverflowError e) {
        //
        // no message in exception (getMessage() returned null)
        //
    }
}
```